**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO**
**CAMPUS ARARAQUARA**

**MYKE ALEXSANDRO BUENO (AQ3032205)**
**ABILIO ADER ROSA DE OLIVEIRA (AQ3032167)**

# Desenvolvimento de Rotor de Código Aberto para Automação de Antenas Direcionais para Estações de Radioamadores

ARARAQUARA
2026

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO**
**CAMPUS ARARAQUARA**

**MYKE ALEXSANDRO BUENO (AQ3032205)**
**ABILIO ADER ROSA DE OLIVEIRA (AQ3032167)**

# Desenvolvimento de Rotor de Código Aberto para Automação de Antenas Direcionais para Estações de Radioamadores

Relatório Técnico submetido ao Instituto Federal de Educação, Ciência e Tecnologia de São Paulo – Campus Araraquara como parte dos requisitos necessários para a obtenção da conclusão do curso Técnico em Mecatrônica.

ARARAQUARA
2026

# RESUMO

Este trabalho apresenta o desenvolvimento de um sistema microcontrolado para o posicionamento angular absoluto de um rotor de antena, visando atender às demandas de precisão e confiabilidade no radioamadorismo. A problemática central reside na perda de referência de posicionamento em motores convencionais após interrupções de energia, o que foi solucionado através da implementação de um encoder absoluto customizado de 8 bits. O protótipo utiliza um disco com codificação em Código Gray, processado por um Arduino Mega Pro Mini, garantindo a integridade dos dados durante as transições de estado e eliminando a necessidade de procedimentos de homing. O sistema conta com um circuito de potência baseado em modulação por largura de pulso (PWM) com o CI 555 para o controle de velocidade e ponte H para reversão do motor. A interface homem-máquina (IHM) é composta por um display LCD 20x4 com fontes customizadas, teclado matricial e sinalização sonora, permitindo a operação manual e o uso de presets geográficos para regiões como EUA, Europa e África através de um algoritmo de trajetória otimizada. Medidas de segurança foram integradas via software e hardware, incluindo um fim de curso instalado no limite de 180° para proteção dos cabos de alimentação. Os resultados demonstraram um sistema de baixo custo com resolução angular de aproximadamente 1,4°, provendo uma solução robusta e intuitiva para o direcionamento de antenas de telecomunicação.

**Palavras-chave:** 1. Código Gray. 2. Encoder Absoluto. 3. Arduino. 4. Controle de Azimute.

# SUMÁRIO

1 INTRODUÇÃO.....06
2 REFERENCIAL TEÓRICO.....07
2.1 Sistemas de Posicionamento: Incremental vs. Absoluto.....08
2.2 Código Gray vs. Binário Puro.....09
2.3 Conceitos básicos de eletrônica.....10
2.4 PWM (Pulse Width Modulation ou Modulação de largura de pulso).....11
2.5 Fundamentos de microcontroladores.....12
2.6 Criação de Esquema Eletrônico.....13
2.7 Desenho técnico e manufatura 3D.....14
2.8 Lógica de funcionamento do firmware do ATmega 2560.....15
3 ANÁLISE EXPERIMENTAL.....16
3.1 Microcontrolador Arduino Mega pro mini 2560.....17
3.2 Desenvolvimento e implementação de driver PWM com CI 555 e IR4427.....18
3.3 Interface Homem-Máquina (IHM).....19
3.4 Sistema de fim de curso para proteção do eixo e cabos.....21
3.5 Sistema de alimentação do projeto.....23
3.6 Módulo relé duplo para acionamento e controle de sentido de rotação do motor.....24
3.7 Desenvolvimento e implementação de emissor e receptor.....25
3.8 Desenvolvimento e implementação de placa de interface.....27
3.9 Desenvolvimento de peças impressas em 3D.....28
3.10 Réplica representativa da antena de 3 seções para onda de 10M.....31
3.11 Estrutura mecânica em andares e organização de componentes.....32
3.11.1 Organização mecânica da estrutura.....32
3.12 Pinagem do microcontrolador e conexão dos periféricos.....37
3.13 Disposição física da montagem.....39
4 RESULTADOS.....44
4.1 Leitura do código Gray e precisão angulares.....44
4.2 Acionamento do motor e controle de sentido.....44
4.3 Interface homem-máquina e modos de operação.....45
4.4 Proteção dos fins de curso e botões manuais.....45
4.5 Discussão dos resultados.....46
5 CONCLUSÃO.....47

6 TRABALHOS FUTUROS .....48
ANEXOS.....51

# 1 INTRODUÇÃO

Este relatório aborda o desenvolvimento de um sistema de controle de azimute para antenas direcionais, integrando microcontroladores, instrumentação e telecomunicações. No radioamadorismo, o apontamento preciso é vital para contatos de longa distância (DX). O projeto resolve a perda de referência comum em sistemas com potenciômetros ou encoders incrementais, propondo um encoder absoluto customizado com Código Gray, que permite a leitura imediata da posição sem necessidade de *homing*.

A justificativa fundamenta-se na busca por alta confiabilidade e baixo custo. Enquanto rotores comerciais possuem preços elevados, o uso de hardware livre, como o Arduino Mega Pro Mini, torna a tecnologia acessível. A relevância técnica reside no uso do Código Gray, que minimiza erros de leitura em transições de estado, garantindo precisão e segurança operacional no direcionamento das antenas.

O objetivo geral é projetar e construir um controlador de rotor com posicionamento automático, manual e feedback absoluto. Os objetivos específicos incluem: (a) desenvolver um encoder absoluto de 8 bits; (b) implementar interface IHM com LCD 20x4 e teclado para coordenadas e presets; (c) elaborar circuito de potência com PWM e inversão via relés; e (d) programar um algoritmo de trajetória limitada (fim de curso em 180º para ambos os lados) e como medida de redundância instalar um fim de curso físico em 180º para ambos os sentidos (H. e A.H).

A metodologia consistiu na simulação do controle de velocidade via CI 555 no EasyEDA, seguida pela confecção do disco gray e montagem do hardware. O software foi desenvolvido em C++ (IDE Arduino) com estrutura de máquinas de estados para os modos de operação. Por fim, testes experimentais validaram a precisão angular e a eficácia das travas de software contra o giro excessivo, protegendo os cabos coaxiais.

# 2 REFERENCIAL TEÓRICO

O desenvolvimento do sistema de posicionamento angular absoluto com encoder de código Gray fundamentou-se em conceitos de eletricidade e eletrônica, microcontroladores, encoders, PWM, interfaces homem-máquina e manufatura aditiva. A Lei de Ohm ($V = R \times I$) rege a relação entre tensão, corrente e resistência, sendo base para o dimensionamento de componentes. Resistores limitam corrente e dividem tensão; capacitores filtram ruídos e suavizam variações; diodos permitem corrente em um único sentido, usados em proteção e retificação. Fontes de alimentação lineares ou chaveadas fornecem as tensões necessárias ao sistema, com parâmetros críticos como regulação de carga e ripple.

PWM (Pulse Width Modulation) é uma técnica que gera sinal digital oscilante, variando o ciclo de trabalho (duty cycle) para controlar a potência média entregue a motores ou LEDs. A frequência deve ser alta o suficiente para evitar vibrações ou tremulação. O microcontrolador ATmega 2560, base do Arduino Mega, integra CPU, memórias flash, SRAM e EEPROM, e periféricos de I/O. A plataforma Arduino, inserida no movimento de hardware livre, reduz custos e barreiras de desenvolvimento.

Encoders rotativos convertem movimento angular em sinais elétricos. O encoder incremental gera pulsos defasados, mas não retém posição após desligamento. O encoder absoluto fornece palavra binária única por posição, mantendo a referência. O código Gray é utilizado para eliminar ambiguidades em transições entre posições consecutivas, pois apenas um bit muda por vez, diferentemente do binário convencional. Para um disco de 8 bits, a resolução é de aproximadamente 1,4°.

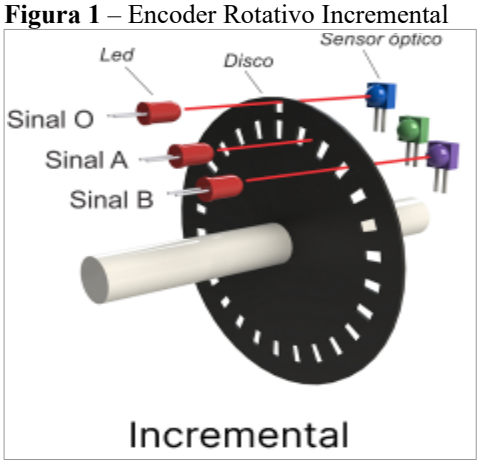
A interface homem-máquina utiliza display LCD 20x4 com protocolo I2C (SDA e SCL), reduzindo o número de pinos, e teclado matricial 4x4 para entrada de comandos. A modelagem 3D paramétrica (Autodesk Inventor) permite projetar peças personalizadas, que são convertidas em G-code por softwares fatiadores (slicer) e fabricadas por impressão FDM com filamento PLA. Esquemas eletrônicos representam a interligação dos componentes, permitindo simulação e documentação do projeto. Esses conceitos integraram a base teórica para o desenvolvimento do protótipo.[1]

1 A integração desses conhecimentos constitui a essência da formação em mecatrônica, unindo engenharia elétrica, mecânica e computação.

## 2.1 Sistemas de Posicionamento: Incremental vs. Absoluto

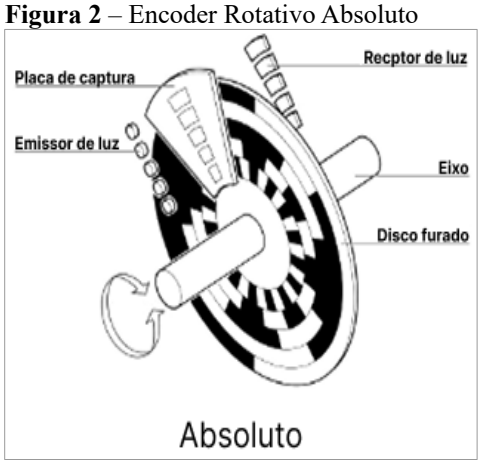
Os encoders rotativos são transdutores eletromecânicos que convertem a posição angular ou o movimento de um eixo em sinais elétricos codificados, sendo amplamente utilizados em sistemas de automação e controle. O encoder incremental (Figura 1) é composto por um disco com ranhuras, um LED emissor e um sensor óptico receptor; à medida que o eixo gira, a passagem intermitente de luz gera pulsos elétricos organizados em dois sinais defasados em 90 graus (A e B), o que permite determinar velocidade, sentido de rotação e posição relativa, embora não retenha a posição após o desligamento do sistema, exigindo referenciamento a cada religamento. Já o encoder absoluto (Figura 2) possui um disco com múltiplas trilhas concêntricas padronizadas, onde cada combinação única de furos (passagem de luz) e ausência de furos (bloqueio) representa um código binário correspondente a uma posição angular específica; dessa forma, o receptor de luz e a placa de captura interpretam instantaneamente a posição atual do eixo, que é mantida mesmo após o desligamento e religamento da alimentação, dispensando qualquer procedimento de referenciamento. A principal diferença entre ambos reside, portanto, na necessidade de referenciamento inicial no encoder incremental e na capacidade de leitura absoluta e imediata do encoder absoluto, sendo o primeiro mais adequado para medição de velocidade e controle de motores de baixo custo, enquanto o segundo é indicado para aplicações que exigem precisão, segurança e retenção de posição, como em robótica e máquinas-ferramenta.[2]

**Figura 1** – Encoder Rotativo Incremental




Fonte: Hitecnologia

**Figura 2** – Encoder Rotativo Absoluto




Fonte: Unb (2018)

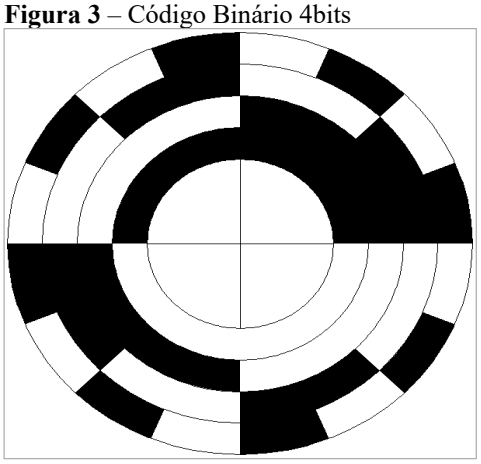
2 Os encoders incrementais são mais econômicos, enquanto os absolutos oferecem maior precisão, sendo indicados para aplicações críticas que exigem posição imediata após religamento.

## 2.2 Código Gray vs. Binário Puro

No código binário convencional (Figura 3), os valores para 16 posições seguem a sequência natural: 0000, 0001, 0010, ..., 1111. O problema é que, ao passar de uma posição para outra, múltiplos bits podem mudar simultaneamente. Na transição da posição 7 (0111) para a posição 8 (1000), todos os quatro bits se alteram ao mesmo tempo. Devido a pequenos desalinhamentos mecânicos, tolerâncias de fabricação do disco ou variações na resposta dos sensores ópticos, esses bits podem não ser lidos exatamente no mesmo instante pelo receptor, gerando leituras temporariamente erradas ou ambíguas, como 0111 → 1111 → 1000. Esse fenômeno compromete seriamente a precisão do encoder, especialmente em altas velocidades de rotação, onde o tempo de leitura de cada bit é extremamente curto.

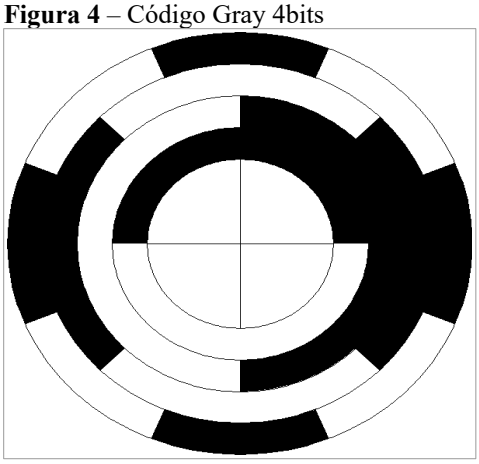
O código Gray (Figura 4) foi desenvolvido para resolver esse problema. Nesse sistema, posições angulares consecutivas diferem em apenas um único bit. Para um disco de 4 bits com 16 posições, a sequência Gray é: 0000, 0001, 0011, 0010, 0110, 0111, 0101, 0100, 1100, 1101, 1111, 1110, 1010, 1011, 1001, 1000. Assim, ao se mover para a posição vizinha, apenas um bit se altera. Mesmo com desalinhamentos, a ambiguidade é eliminada, pois a mudança de um único bit é interpretada de forma confiável. A principal desvantagem do código Gray é que ele não é diretamente compatível com operações aritméticas, sendo necessária sua conversão para binário convencional por meio de uma tabela ou circuito lógico antes que o valor possa ser processado por um microcontrolador. [3]

**Figura 3** – Código Binário 4bits



Fonte: Hitecnologia

**Figura 4** – Código Gray 4bits



Fonte: Hitecnologia

3 Enquanto o código binário é mais intuitivo para operações matemáticas, o código Gray é amplamente preferido em encoders absolutos de alta velocidade devido à sua imunidade a erros de leitura durante transições entre posições adjacentes.

## 2.3 Conceitos básicos de eletrônica

O projeto de qualquer sistema eletrônico fundamenta-se em conceitos básicos de eletricidade e eletrônica. A tensão elétrica (medida em volts) é a diferença de potencial que impulsiona os elétrons através de um condutor, enquanto a corrente elétrica (medida em ampères) é o fluxo desses elétrons. A resistência elétrica (medida em ohms) é a oposição a esse fluxo, e a relação entre tensão, corrente e resistência é governada pela Lei de Ohm: $V = R \times I$. A potência elétrica dissipada em um componente (medida em watts) pode ser calculada como $P = V \times I$ ou $P = R \times I^2$.

Os componentes eletrônicos passivos mais comuns incluem resistores, capacitores e indutores. Os resistores limitam a corrente e dividem tensões, sendo especificados por seu valor de resistência e potência máxima dissipável. Capacitores armazenam energia na forma de campo elétrico, suavizando variações de tensão e filtrando ruídos; são caracterizados por sua capacitância (em farads) e tensão máxima de trabalho. Indutores armazenam energia na forma de campo magnético e são usados em filtros. Componentes ativos, como diodos e transistores, controlam o fluxo de corrente. Diodos permitem a passagem de corrente em apenas um sentido, sendo utilizados em retificadores e proteções contra inversão de polaridade. Transistores atuam como chaves ou amplificadores.

As fontes de alimentação são responsáveis por fornecer as tensões e correntes necessárias ao sistema. Uma fonte linear utiliza transformador, ponte retificadora, capacitores de filtro e regulador de tensão. Embora produza baixo ruído, é ineficiente para grandes correntes. Uma fonte chaveada opera por comutação em alta frequência, resultando em maior eficiência energética e menor tamanho, mas exige controle cuidadoso de interferências. Parâmetros críticos de uma fonte incluem tensão nominal, corrente máxima, regulação de carga, ripple e isolamento. O dimensionamento correto envolve calcular a corrente consumida por todos os subsistemas e adicionar margem de segurança de 20% a 30% para picos de partida. Capacitores de desacoplamento (tipicamente 100 nF) devem ser posicionados próximos aos circuitos integrados para suprimir ruídos de alta frequência, enquanto capacitores eletrolíticos de maior valor atuam como reservatórios de energia para demandas transitórias. O conhecimento desses conceitos é fundamental para projetar, dimensionar e diagnosticar qualquer sistema eletrônico.[4]

4 A Lei de Ohm permite calcular tensões, correntes e resistências em qualquer circuito. O correto dimensionamento de resistores e capacitores evita instabilidades e danos aos componentes sensíveis.

## 2.4 PWM (Pulse Width Modulation ou Modulação de largura de pulso)

PWM (Pulse Width Modulation) é uma técnica amplamente utilizada em eletrônica e mecatrônica para obter resultados analógicos através de meios digitais. Diferentemente de uma tensão analógica verdadeira, que pode assumir infinitos valores entre dois limites, um sinal PWM é essencialmente digital, oscilando entre dois níveis discretos: HIGH (ligado, tipicamente 5V ou 3,3V) e LOW (desligado, 0V). A chave para seu funcionamento está na variação da proporção de tempo que o sinal permanece em HIGH em relação ao período total de oscilação. Essa proporção é chamada de ciclo de trabalho (duty cycle) e é expressa em porcentagem. Um ciclo de trabalho de 0% significa que o sinal está permanentemente em LOW (desligado), enquanto 100% significa permanentemente em HIGH (ligado). Um ciclo de 50% indica que o sinal fica metade do tempo em HIGH e metade em LOW. Quando esse padrão de chaveamento é repetido rápido o suficiente, a carga conectada (como um motor DC ou um LED) não consegue responder às oscilações individuais devido à sua inércia mecânica ou à persistência da visão humana. O motor, por exemplo, comporta-se como se estivesse recebendo uma tensão contínua variável proporcional ao ciclo de trabalho: com 50% do ciclo em HIGH, a tensão média efetiva é de 2,5V (metade de 5V), entregando metade da potência nominal.

No contexto da mecatrônica, o PWM é a técnica padrão para controle de velocidade de motores DC, pois permite variar a potência entregue sem a perda energética característica de métodos lineares (como resistores variáveis). Além disso, é utilizada para controle de servomotores, dimerização de LEDs, geração de sinais de áudio e até mesmo em fontes chaveadas. A frequência do sinal PWM é um parâmetro importante: frequências muito baixas (abaixo de 100 Hz) podem causar tremulação perceptível em LEDs ou vibração em motores, enquanto frequências muito altas (acima de 20 kHz) podem exigir componentes de chaveamento mais rápidos e gerar interferências eletromagnéticas. A plataforma Arduino, por exemplo, gera PWM em frequências típicas de 490 Hz ou 980 Hz em seus pinos digitais configurados com a função analogWrite(), sendo adequadas para a maioria das aplicações de controle de motores e LEDs. No presente projeto, a placa PWM desenvolvida com dois CI 555 e um driver IR4427 foi responsável por gerar o sinal PWM que controla a velocidade do motor de giro do eixo, permitindo testar o encoder Gray em diferentes rotações e verificar a confiabilidade da leitura óptica em condições variadas.[5]

5 A frequência do PWM influencia diretamente a ondulação da corrente no motor: frequências mais altas reduzem a ondulação (ripple), mas aumentam as perdas por comutação nos transistores e no driver.

## 2.5 Fundamentos de microcontroladores

Um microcontrolador é um circuito integrado que contém em um único chip todos os elementos essenciais de um sistema computacional: processador (CPU), memórias e periféricos de entrada e saída. Diferentemente de um microprocessador convencional, que requer componentes externos adicionais, o microcontrolador é projetado para operar de forma autônoma em sistemas embarcados, onde espaço físico, consumo de energia e custo são fatores críticos.

A arquitetura típica inclui uma CPU para execução de instruções aritméticas e lógicas, memória flash para armazenamento permanente do programa, memória SRAM para dados temporários, e memória EEPROM para retenção de informações mesmo após desligamento. Os periféricos de I/O incluem portas digitais (GPIO), conversores analógico-digitais (ADC), temporizadores, geradores de PWM, e interfaces de comunicação serial como UART, I2C e SPI.

Os microcontroladores são classificados por arquitetura de instruções (CISC ou RISC), largura de barramento (8, 16 ou 32 bits), frequência de operação (1 MHz a 300 MHz), capacidade de memória e tensão de alimentação (3,3V ou 5V). A família AVR, da Microchip, é amplamente utilizada em projetos educacionais devido à sua arquitetura RISC eficiente, que executa a maioria das instruções em um único ciclo de clock. Um microcontrolador típico de 8 bits opera a 16 MHz e possui dezenas de pinos de I/O digital e múltiplas entradas analógicas.

A programação de microcontroladores pode ser feita em linguagem C/C++ ou Assembly, frequentemente por meio de plataformas de hardware livre como o Arduíno, que abstraem detalhes de baixo nível e fornecem bibliotecas prontas. O hardware livre disponibiliza esquemáticos e códigos publicamente, reduzindo barreiras de entrada e permitindo que estudantes e pequenos empreendedores desenvolvam protótipos com baixo investimento. O ciclo de desenvolvimento envolve escrita do código em uma IDE, compilação e carregamento do programa na memória flash via programador serial. Os periféricos de temporização e interrupções são particularmente importantes em aplicações que exigem resposta em tempo real, permitindo que o microcontrolador execute rotinas prioritárias sem perder eventos externos.[6]

6 A escolha da arquitetura do microcontrolador deve considerar o equilíbrio entre consumo energético, capacidade de processamento, número de pinos de I/O e periféricos integrados, adequando-se às necessidades específicas de cada sistema embarcado.

## 2.6 Criação de Esquema Eletrônico

O desenvolvimento de qualquer sistema eletrônico requer a representação gráfica de seus componentes e conexões por meio de um esquema elétrico (ou esquemático). Este documento é a "planta baixa" do circuito, permitindo ao projetista visualizar a interligação entre os elementos antes mesmo da montagem física. No contexto deste trabalho, a elaboração dos esquemas foi fundamental para projetar a placa PWM, a placa de sensores (LEDs IR e fototransistores), a interface com o microcontrolador e o sistema de relés. Um esquema eletrônico utiliza símbolos padronizados para representar componentes como resistores, capacitores, diodos, transistores, circuitos integrados (CIs) e conectores. As linhas contínuas representam condutores elétricos (trilhas ou fios), enquanto pontos de junção indicam conexões elétricas entre eles. Cada componente recebe uma referência alfanumérica (ex.: R1 para resistor, C1 para capacitor, U1 para CI 555) que o identifica unicamente na lista de materiais (BOM – Bill of Materials).

Atualmente, o projeto de esquemas eletrônicos é feito com softwares de CAD (Computer-Aided Design) específicos, como KiCad, EAGLE, Altium Designer ou Fritzing. Essas ferramentas oferecem bibliotecas de componentes, recursos de verificação de erros (como conexões faltantes ou curtos-circuitos) e a capacidade de exportar o esquema para o layout da placa de circuito impresso (PCI). Na mecatrônica, o esquema eletrônico é a ponte entre o conceito teórico e a realização física. Ele permite a simulação do comportamento do circuito com ferramentas como SPICE, identificando falhas de projeto antes da soldagem; documenta o projeto de forma a facilitar a manutenção e o reaproveitamento; e mostra como os subsistemas (sensores, atuadores, microcontrolador, fonte de alimentação) se conectam, evitando erros de montagem. No esquema da placa de sensores (Figura 17), os oito LEDs IR com resistores de 150 Ω e os oito fototransistores com resistores de pull-up de 10 kΩ foram dispostos de forma organizada, facilitando o roteamento das conexões para o conector DB e o microcontrolador. A identificação dos sinais S1 a S8 e das alimentações VCC e GND seguiu as convenções adotadas nos demais esquemas, garantindo consistência em todo o projeto. Assim, a criação de esquemas eletrônicos não é apenas um requisito formal, mas uma etapa essencial do processo de desenvolvimento em mecatrônica, assegurando que o protótipo funcione conforme o planejado e possa ser reproduzido por terceiros.[7]

7 A utilização de uma nomenclatura clara e padronizada nos esquemas (R1, C1, U1, S1, VCC, GND) é essencial para que o projeto possa ser interpretado corretamente por outros técnicos ou engenheiros, garantindo a reprodutibilidade do sistema.

## 2.7 Desenho técnico e manufatura 3D

O desenvolvimento de sistemas mecatrônicos frequentemente exige a fabricação de peças personalizadas para suporte, alinhamento e fixação. Nesse contexto, o desenho técnico assistido por computador (CAD) e a manufatura aditiva (impressão 3D) tornaram-se ferramentas indispensáveis. O desenho técnico é a linguagem gráfica da engenharia, permitindo representar com precisão as dimensões, formas geométricas, tolerâncias e acabamentos de uma peça. Diferentemente de um desenho artístico, o desenho técnico não admite ambiguidades: cada linha, cota e símbolo possui significado exato, garantindo que a peça possa ser fabricada por terceiros sem explicações adicionais.

A modelagem 3D é a evolução digital do desenho técnico. Utilizando softwares CAD paramétricos como Autodesk Inventor, SolidWorks ou Fusion 360, o projetista cria um modelo tridimensional virtual da peça, definindo extrudes, revoluções, cortes e operações lógicas a partir de esboços 2D. A modelagem paramétrica é particularmente poderosa porque as dimensões são armazenadas como parâmetros que podem ser alterados a qualquer momento, atualizando automaticamente todo o modelo. Isso permite explorar variações de projeto sem redesenhar a peça do zero. No presente trabalho, o Autodesk Inventor foi utilizado para modelar o disco Gray de 8 bits, o suporte para LEDs (emissores na parte superior e receptores na inferior), as regulagens menor e maior para alinhamento fino dos sensores, e os componentes dos pés reguladores. Cada modelo foi exportado no formato STL, que representa a superfície da peça como uma malha de triângulos.

O fatiador (slicer) é o software que converte o modelo 3D em instruções de movimento para a impressora 3D, gerando um arquivo G-code. O processo consiste em "fatiar" o modelo em centenas de camadas horizontais de espessura constante, calculando para cada camada o perímetro externo (contorno) e o preenchimento interno (infill). Parâmetros críticos configuráveis no slicer incluem: altura da camada (relacionada à resolução vertical e ao tempo de impressão); temperatura de extrusão (depende do filamento utilizado); temperatura da mesa aquecida (para evitar empenamento); velocidade de impressão; densidade e padrão do preenchimento interno; número de camadas sólidas superiores e inferiores (essencial para garantir rigidez e opacidade); e geração de suportes para partes com balanço excessivo. Após o fatiamento, o G-code é transferido para a impressora e as peças são fabricadas camada por camada por deposição de material fundido (FDM).[8]

8 O número de camadas sólidas superiores e inferiores influencia diretamente a opacidade e a rigidez da peça, sendo um parâmetro crítico em aplicações que exigem bloqueio total de passagem de luz.

## 2.8 Lógica de funcionamento do firmware do ATmega 2560

Firmware é o software de baixo nível armazenado permanentemente em memória não volátil (flash, ROM ou EEPROM) de um dispositivo eletrônico, sendo responsável por controlar suas operações fundamentais. Diferentemente de softwares aplicativos executados em sistemas operacionais genéricos, que contam com camadas de abstração de hardware, o firmware é projetado especificamente para o hardware no qual reside, interagindo diretamente com registradores, periféricos, temporizadores e interrupções do microcontrolador. Em sistemas embarcados – dispositivos computacionais dedicados a funções específicas como eletrodomésticos, automóveis, equipamentos industriais, dispositivos médicos e eletrônicos de consumo – o firmware é o elemento central que traduz o comportamento desejado em ações físicas sobre sensores e atuadores.

O desenvolvimento de firmware segue um ciclo iterativo bem definido: definição de requisitos; escolha do microcontrolador e ambiente de desenvolvimento; codificação em linguagem de alto nível (C ou C++ é a escolha mais comum) utilizando uma IDE como Arduino IDE, PlatformIO ou VS Code; compilação do código-fonte para código de máquina específico do microcontrolador; carga do binário na memória flash via programadores ou bootloaders; depuração utilizando breakpoints, observação de variáveis e análise de temporização; e testes de validação em condições reais ou simuladas. Máquinas de estado finito são amplamente empregadas para organizar a lógica de sistemas que podem operar em diferentes modos (inicialização, operação normal, emergência, calibração), tornando o código mais previsível e fácil de manter.

Com a popularização do hardware livre e do código aberto, o desenvolvimento de firmware tornou-se significativamente mais acessível. Projetistas podem utilizar bibliotecas já testadas para protocolos complexos (TCP/IP, CAN, Modbus), drivers para sensores comuns e algoritmos de controle como PID. Plataformas como Arduino reduzem a barreira de entrada ao abstrair detalhes de baixo nível e fornecer uma API simples e homogênea entre diferentes microcontroladores. A colaboração através de repositórios públicos acelera o ciclo de inovação, permitindo que soluções para problemas comuns sejam reutilizadas e aprimoradas pela comunidade. A documentação adequada do firmware – incluindo comentários, descrição da máquina de estados e especificação dos protocolos – é essencial para garantir que o sistema possa ser mantido e estendido por outros desenvolvedores.[9]

9 A diferença fundamental entre firmware e software aplicativo é o grau de dependência do hardware: o firmware opera diretamente sobre registradores e periféricos do microcontrolador, enquanto o software aplicativo depende de um sistema operacional para intermediar o acesso aos recursos.

# 3 ANÁLISE EXPERIMENTAL

O sistema integrado – encoder absoluto de código Gray com 8 bits (256 posições), placa PWM com CI 555 e driver IR4427, relé duplo, IHM com LCD 20x4 e teclado 4x4, botões manuais, reset, fins de curso e diodos – foi validado no laboratório do IFSP Campus Araraquara. A estrutura de três andares organizou a fonte ATX 500W no térreo, sensores, ATmega 2560 e relés no nível intermediário, e a IHM no superior.

O objetivo central foi verificar a leitura confiável do código Gray em diferentes velocidades, a resposta correta aos comandos manuais e programados, respeitando os limites dos fins de curso para proteger o encoder e o cabo da antena. Avaliou-se também a precisão angular, a estabilidade da fonte ATX, a eficácia da dupla regulagem 3D para alinhamento dos pares LED IR/fototransistor, e a atuação dos diodos D1 e D2 no corte da função dos botões manuais.

Os experimentos foram organizados em três fases. Primeira: testes individuais da placa PWM e do relé duplo, verificando a onda modulada em diferentes ciclos de trabalho (0% a 100%) e a inversão de polaridade do motor, monitorando corrente e formas de onda. Segunda: avaliação do subsistema óptico – placas emissoras (LEDs IR com resistores de 150 Ω) e receptora (fototransistores com pull-up de 10 kΩ) alinhadas por regulagens impressas em PLA preto (Elegoo Neptune 4 Pro, 5 camadas). O disco Gray foi girado em velocidades crescentes, com os 8 bits (S1 a S8) exibidos no LCD e no monitor serial, comparando a leitura com a sequência teórica das 256 posições.

Terceira fase: testes integrados. O ATmega 2560 recebeu os sinais dos fototransistores, interpretou o código Gray, converteu para posição angular (0° a 359°, passo de 1,4°), exibiu os dados no LCD e acionou relés e PWM para mover o motor até a posição desejada. Os fins de curso (SW1 e SW2) foram conectados em série com a alimentação do motor, com diodos D1 e D2 no meio do cabo cortando apenas o comando manual naquele sentido. Testaram-se velocidades de 1, 2 e 3 RPM para identificar a rotação máxima sem erros de leitura e visando preservar a integridade da antena.[10]

10 Foram descartadas medições cujo desvio padrão excedeu 5% do valor médio, repetindo-se o ensaio imediatamente para evitar a influência de fatores transitórios, como oscilações térmicas ou ruídos na alimentação. Esse procedimento garantiu a confiabilidade estatística dos resultados apresentados na tabela deste capítulo.

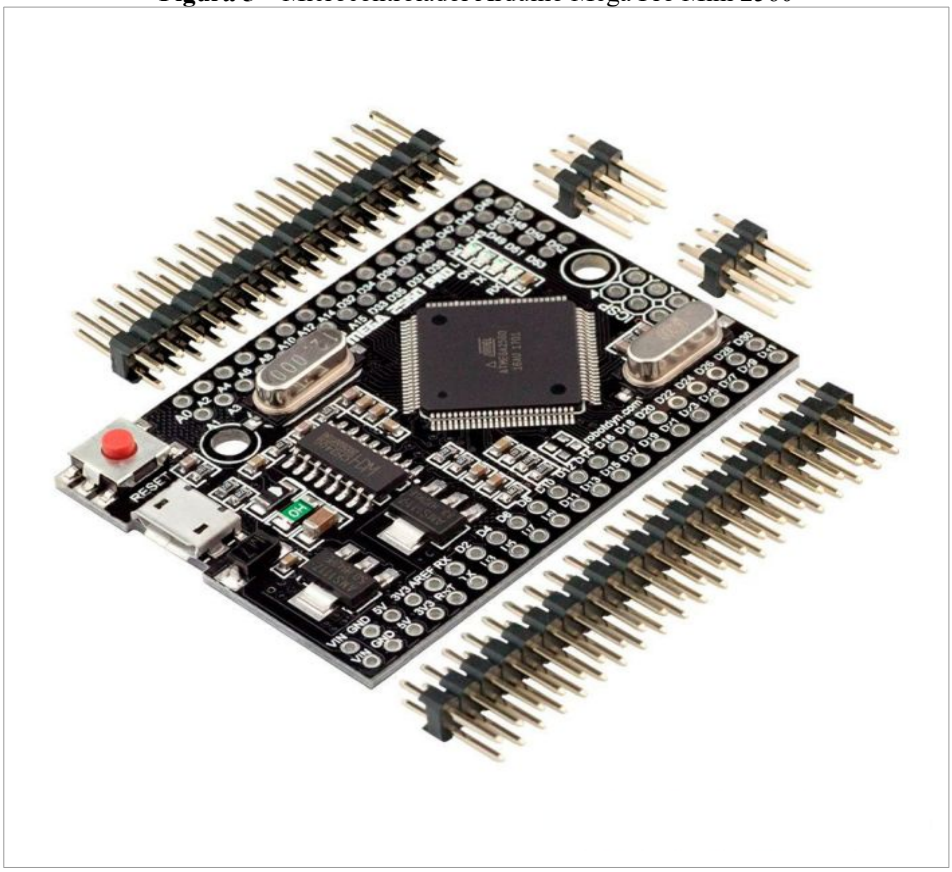
### 3.1 Microcontrolador Arduino Mega pro mini 2560

O ATmega 2560 é um microcontrolador da família AVR, conhecido por equipar o Arduino Mega, sendo ideal para projetos que exigem muitos pinos e grande capacidade de memória. A Figura 5 apresenta sua disposição típica.

O ATmega 2560 opera a 16 MHz e possui 256 KB de flash, 8 KB de SRAM e 4 KB de EEPROM. Dispõe de 86 pinos de I/O digital, 16 entradas analógicas de 10 bits, 4 portas seriais (UART) e suporte a I²C e SPI. Sua alimentação é de 4,5V a 5,5V, sendo comumente utilizado em sistemas de 5V. Pode ser programado via Arduino IDE com programador externo ou bootloader.

Neste trabalho, o ATmega 2560 foi escolhido pelo elevado número de pinos para leitura simultânea dos sensores, processamento e exibição da posição angular. Sua memória permite conversão entre códigos binário e Gray, além da interface com displays e atuadores.[11]

**Figura 5** – Microcontrolador Arduino Mega Pro Mini 2560



Fonte: Dreletronico

11 O ATmega 2560 também pode ser encontrado em versões de 8 MHz para operação com tensão de 3,3V, sendo necessário atentar à compatibilidade de níveis lógicos ao integrá-lo com sensores e periféricos.

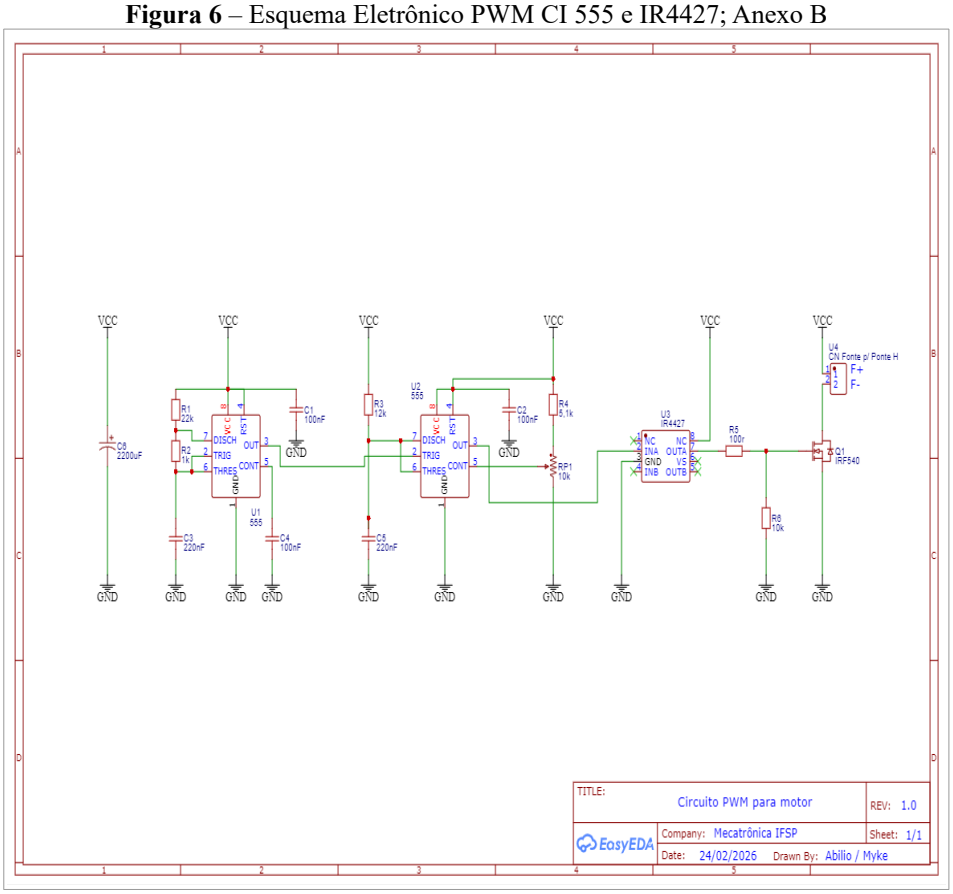
### 3.2 Desenvolvimento e implementação de driver PWM com CI 555 e IR4427

Para integrar o disco Gray ao projeto, foi desenvolvida uma placa de modulação por largura de pulso (PWM) capaz de controlar a velocidade de diferentes motores. A Figura 6 apresenta o esquemático completo.

O circuito utiliza dois CI 555 e um driver IR4427. O primeiro CI 555 gera uma onda quadrada com frequência fixa, enquanto o segundo ajusta o ciclo de trabalho do sinal PWM. O IR4427 amplifica a corrente para acionar os MOSFETs da ponte H que alimentam o motor. A placa conta com capacitores de filtro (2200 µF e 100 nF), resistores de polarização (22k, 10k, 12k e 5,1k) e terminais F+ e F- para a fonte de potência.

No contexto deste trabalho, a placa PWM foi projetada para acionar o motor que gira o disco Gray transformando o mesmo em um encoder absoluto, permitindo simular diferentes velocidades de rotação e validar a leitura correta do código Gray em condições variadas de operação.[12]

**Figura 6** – Esquema Eletrônico PWM CI 555 e IR4427; Anexo B




Fonte: Elaborado pelo autor

12 O driver IR4427 foi escolhido por sua alta velocidade de comutação (até 60 ns) e capacidade de corrente de pico de 2,5 A, sendo compatível com MOSFETs de potência utilizados em pontes H para motores de até 500 W.

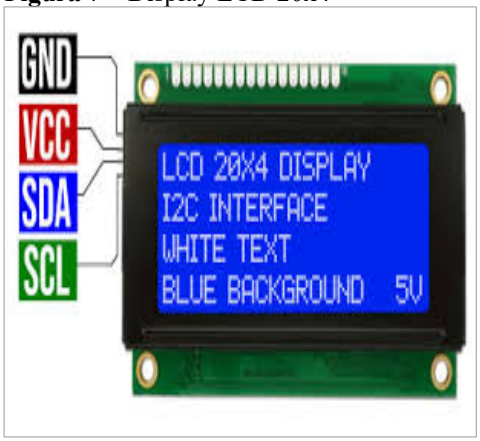
### 3.3 Interface Homem-Máquina (IHM)

Como Interface Homem-Máquina (IHM) do projeto, foram utilizados um display LCD 20x4, um teclado matricial 4x4 e botões físicos para controle manual, permitindo ao operador visualizar informações e enviar comandos ao sistema de forma intuitiva. Essa combinação de dispositivos torna a interação com o encoder mais acessível e dispensa o uso contínuo de um computador para monitoramento. Além disso, a IHM foi projetada para ser de fácil compreensão mesmo para operadores com pouco conhecimento técnico.

O display LCD 20x4 é capaz de exibir 20 caracteres por linha em 4 linhas simultaneamente, sendo utilizado para mostrar em tempo real a posição angular do encoder, a velocidade do motor e o código Gray lido. O modelo possui interface I2C integrada, exigindo apenas quatro conexões (GND, VCC, SDA e SCL), o que simplifica a ligação com o microcontrolador e reduz a complexidade da fiação. A alimentação é de 5V, e o display possui fundo azul com caracteres brancos, garantindo boa legibilidade mesmo em condições de pouca luminosidade. A Figura 7 apresenta o display LCD 20x4 com sua interface I2C.

O teclado matricial 4x4 é composto por 16 teclas dispostas em 4 linhas e 4 colunas, com os seguintes caracteres: 1, 2, 3, A, 4, 5, 6, B, 7, 8, 9, C, *, 0, #, D. Através dele, o usuário pode inserir valores de referência, como a posição desejada para o eixo, acessar favoritos, bem como navegar pelos menus da interface e executar funções específicas do sistema, como iniciar uma rotina automática de teste ou calibrar o encoder. A Figura 8 mostra o teclado matricial utilizado no projeto.[13]

**Figura 7** – Display LCD 20x4




Fonte: Eletrogate

**Figura 8** – Teclado Matricial 4x4




Fonte: Eletrogate

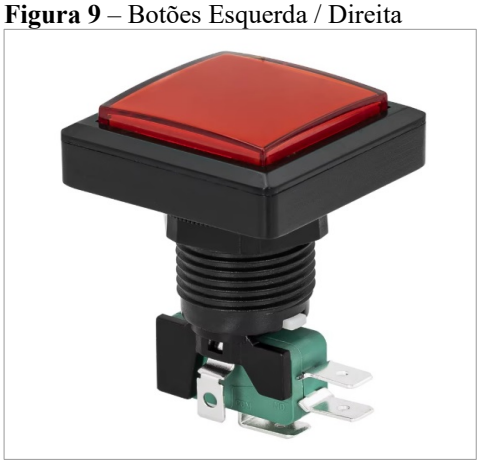
13 O uso de interface I2C no display LCD reduz significativamente o número de pinos do microcontrolador necessários para operação, liberando recursos para a leitura dos encoders e dos botões de controle.

Além do teclado, foram instalados botões de acionamento manual para controle direto do motor. Dois botões dedicados permitem o giro à esquerda e o giro à direita do eixo de forma incremental e precisa, sem necessidade de programar valores via teclado. Esses botões são úteis para ajustes finos, testes de validação e movimentação manual durante a calibração do sistema. A Figura 9 ilustra o botão de giro utilizado.

Também foi implementado um botão de reset, que reinicia o sistema e reorienta o código Gray da posição atual, permitindo ao operador reiniciar a leitura do encoder a qualquer momento. Este botão é essencial para situações de erro ou para iniciar uma nova sequência de testes sem desligar a alimentação do equipamento. A Figura 10 apresenta o botão de reset.

No contexto deste trabalho, a combinação do display LCD 20x4, do teclado matricial e dos botões físicos de giro e reset permite uma interação completa e autônoma com o sistema de encoder Gray, dispensando o uso de computador para monitoramento e controle. O display exibe continuamente os dados do encoder, enquanto o teclado e os botões possibilitam o envio de comandos em tempo real, facilitando a validação experimental e o diagnóstico.

**Figura 9** – Botões Esquerda / Direita



Fonte: Eletronik LV

**Figura 10** – Botão de RESET



Fonte: Eletroparts

A integração entre display, teclado e botões físicos proporciona ao operador três níveis distintos de interação: visualização contínua dos dados, inserção de comandos numéricos e controle manual direto. Essa redundância de interfaces aumenta a robustez do sistema, permitindo que o usuário escolha o método mais adequado para cada situação de operação ou manutenção. Dessa forma, o sistema de encoder Gray torna-se versátil, intuitivo e adequado tanto para bancadas de laboratório quanto para aplicações industriais de pequeno porte.[14]

14 Botões de giro e reset são conectados diretamente ao ATmega 2560 com resistores de pull-up internos.

### 3.4 Sistema de fim de curso para proteção do eixo e cabos

Para evitar danos mecânicos ao sistema, ao encoder e ao cabo da antena, foram instalados fins de curso nos limites máximos de rotação do eixo, um para o giro à esquerda (SW1) e outro para o giro à direita (SW2). Esses componentes atuam como chaves de limite físico, interrompendo o acionamento do motor quando o eixo atinge os extremos permitidos, prevenindo colisões, desalinhamentos e o rompimento ou torção excessiva do cabo da antena.

Os fins de curso utilizados possuem especificações nominais de 12A de corrente e 125 V de tensão. Além disso, foram empregados dois diodos (D1 e D2) posicionados no meio do cabo de alimentação dos botões manuais (esquerda e direita). Cada diodo atua cortando a alimentação do respectivo botão de giro quando o fim de curso correspondente é acionado, impedindo que o comando manual chegue ao motor além do limite seguro. A Figura 11 apresenta o fim de curso, enquanto as Figuras 12 e 13 detalham o esquema elétrico e mecânico da proteção.[15]

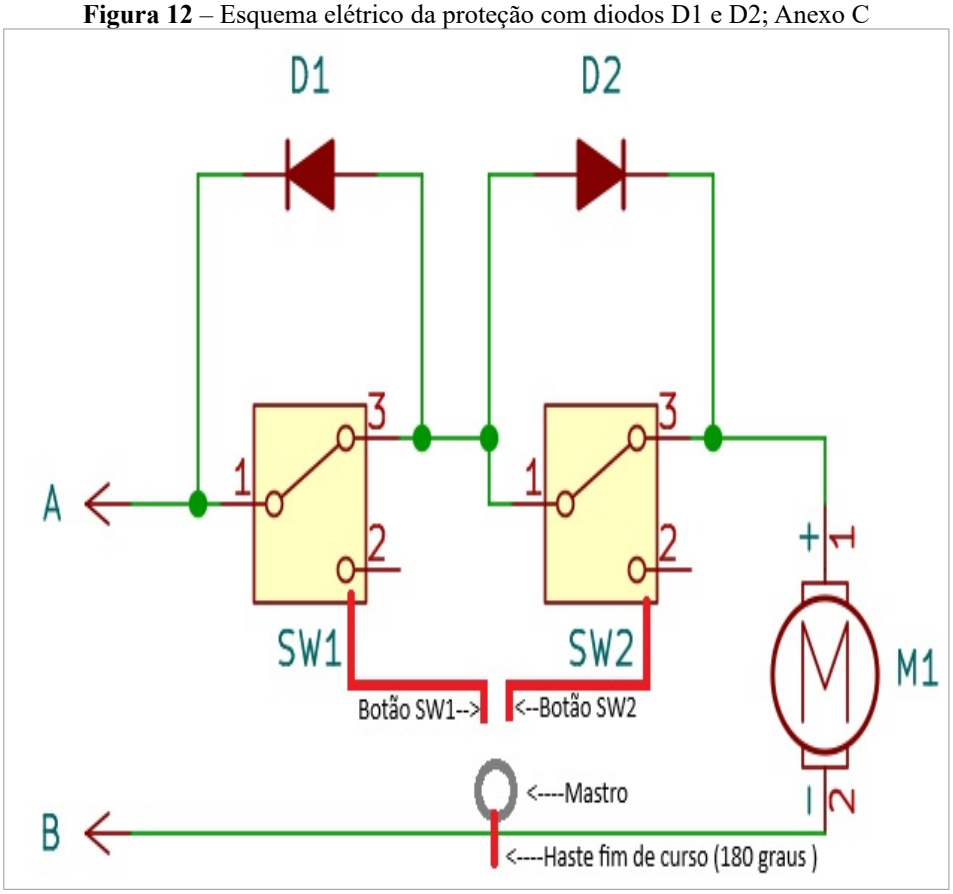
**Figura 11** – Fim de curso utilizado (SW) para os limites de giro (esquerda e direita)




Fonte: Nobrelar

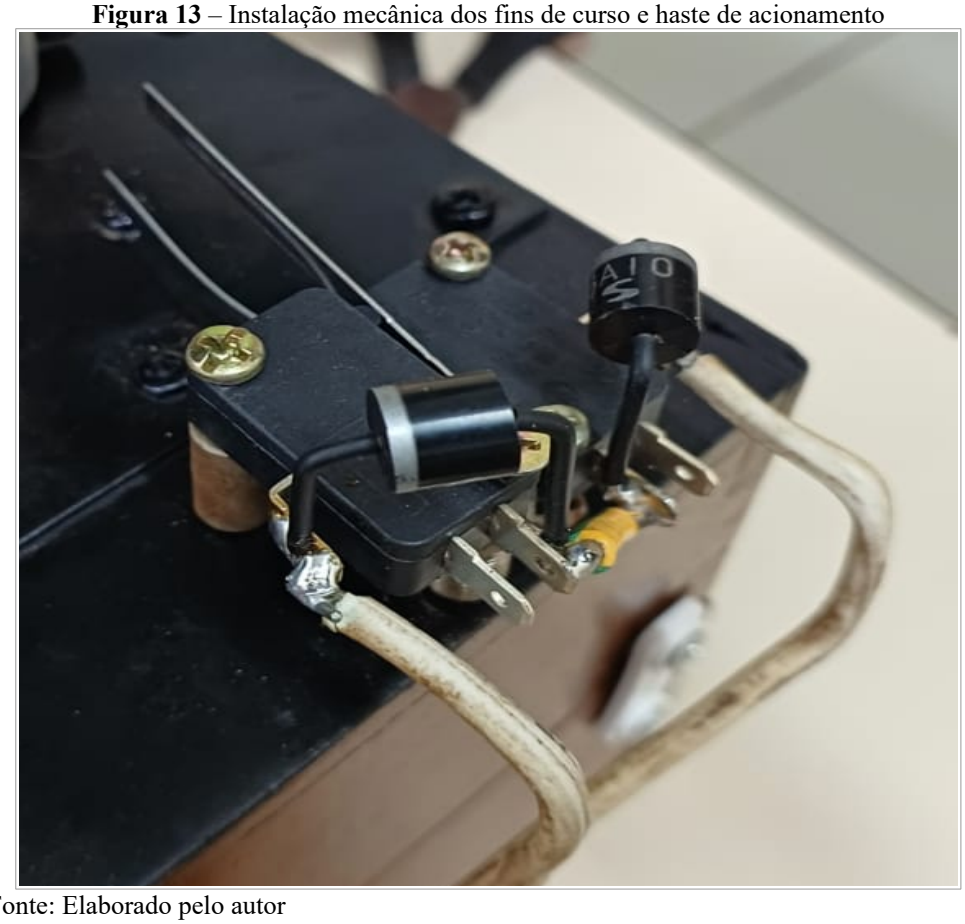
15 O posicionamento dos diodos no meio do cabo garante que o corte da alimentação ocorra de forma localizada, afetando apenas o comando manual sem interferir em outras partes do circuito.

**Figura 12** – Esquema elétrico da proteção com diodos D1 e D2; Anexo C




Fonte: Elaborado pelo autor

**Figura 13** – Instalação mecânica dos fins de curso e haste de acionamento



Fonte: Elaborado pelo autor

### 3.5 Sistema de alimentação do projeto

Para alimentar todos os componentes do projeto, foi utilizada uma fonte ATX 500W Blucase, adaptada para fornecer as diferentes tensões necessárias ao sistema. A Figura 14 apresenta a fonte utilizada.

A fonte disponibiliza barramentos de +12V, +5V e +3,3V, além do sinal PS-ON. O +12V alimentou o motor e a placa PWM; o +5V alimentou o microcontrolador ATmega 2560, display LCD 20x4, teclado matricial e sensores ópticos; o +3,3V ficou disponível para sensores de baixa tensão. Com 500W, a fonte suporta picos de corrente do motor sem quedas significativas.

A adaptação consistiu no desencapsulamento do conector ATX, identificação dos fios por cor (amarelo: +12V, vermelho: +5V, laranja: +3,3V, preto: GND, verde: PS-ON) e soldagem direta aos barramentos. Foi instalado um botão de acionamento que conecta o PS-ON ao GND momentaneamente, permitindo ligar e desligar a fonte conforme a necessidade do operador, aumentando a segurança e a eficiência energética do sistema.[16]

**Figura 14** – Fonte ATX 500W BLUECASE



Fonte: BLUECASE

16 O botão de acionamento da fonte ATX foi conectado em paralelo ao jumper PS-ON/GND, permitindo desligamento total do sistema quando não está em uso, evitando consumo em modo stand-by.

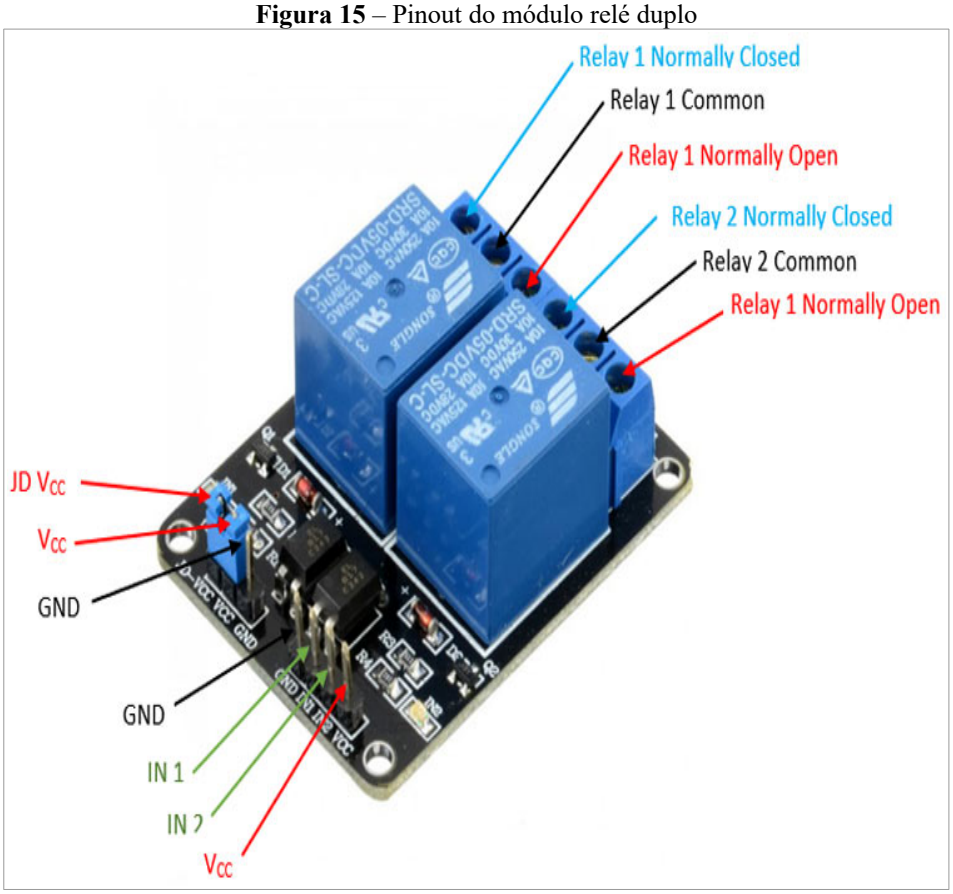
### 3.6 Módulo relé duplo para acionamento e controle de sentido de rotação do motor

Para acionar o motor e controlar seu sentido, foi utilizado um módulo relé duplo, controlado pelo ATmega 2560. O relé acionado determina a polaridade da tensão aplicada ao motor, definindo o giro para esquerda ou direita. A Figura 15 apresenta o pinout do módulo.

O módulo possui dois relés independentes, cada um com terminais NC, COM e NO. Para o controle do sentido, foi implementada uma ponte H com relés, onde o motor é conectado entre os terminais COM. Quando o relé 1 é acionado e o relé 2 desacionado, o motor gira para a direita. Quando o relé 2 é acionado e o relé 1 desacionado, a rotação é invertida para a esquerda.

O módulo recebe comandos via teclado matricial ou botões manuais. O microcontrolador ativa o relé correspondente, garantindo o giro desejado até que um fim de curso seja acionado ou o comando interrompido. A abordagem mostrou-se simples e eficaz, suportando até 10A por relé, compatível com o motor utilizado.[17]

**Figura 15** – Pinout do módulo relé duplo




Fonte: Masterwall

17 O módulo relé duplo apresenta isolamento óptico entre a lógica de controle e a bobina, reduzindo interferências eletromagnéticas no microcontrolador.

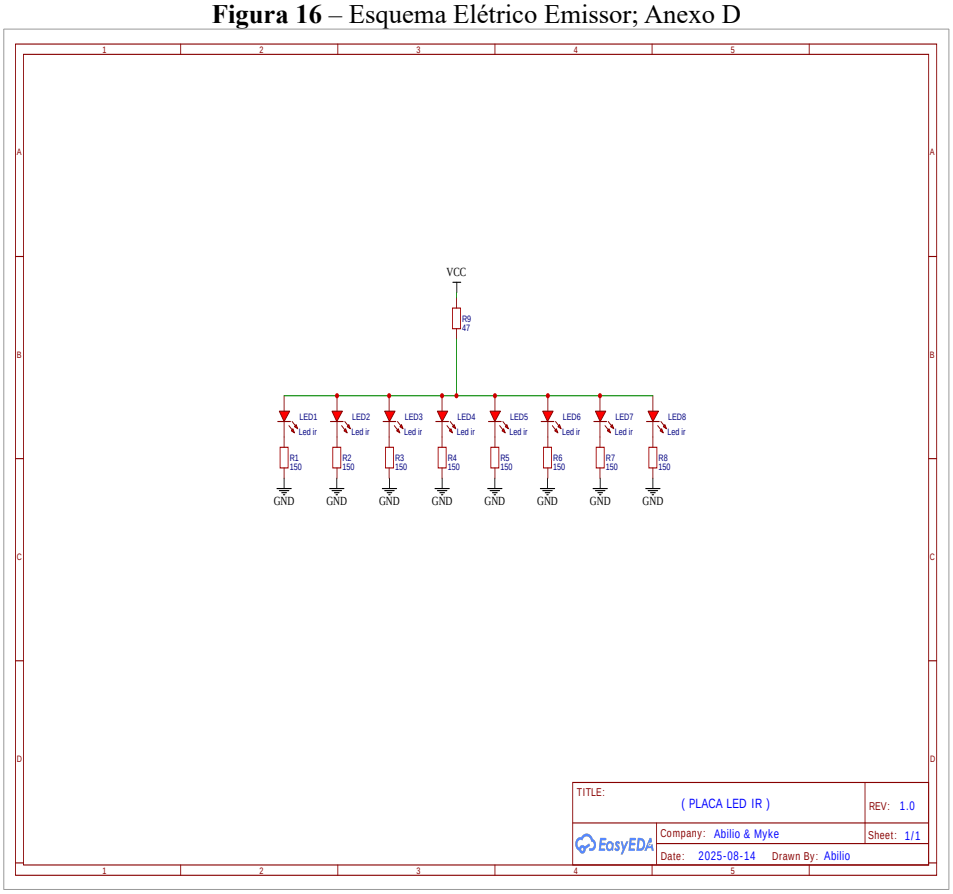
### 3.7 Desenvolvimento e implementação de emissor e receptor

Para realizar a leitura óptica do disco codificado em código Gray, foram desenvolvidas duas placas dedicadas: uma placa emissora e uma placa receptora. A Figura 16 apresenta o esquema da placa emissora, enquanto a Figura 17 mostra a placa receptora.

A placa emissora é composta por oito LEDs infravermelhos (IR), cada um com resistor limitador de corrente de 150Ω, dispostos de forma alinhada às trilhas do disco Gray. O conjunto é alimentado por VCC e possui um resistor de 47Ω para proteção geral do circuito.

A placa receptora utiliza oito fototransistores correspondentes, cada um com resistor de pull-up de 10kΩ, gerando os sinais digitais S1 a S8. Quando a luz IR passa por um furo do disco, o fototransistor conduz e a saída vai a LOW (0 lógico); quando bloqueado, a saída fica em HIGH (1 lógico). Esse conjunto de 8 bits forma o código Gray que é lida pelo microcontrolador ATmega 2560, permitindo determinar com precisão a posição angular do eixo.[18]

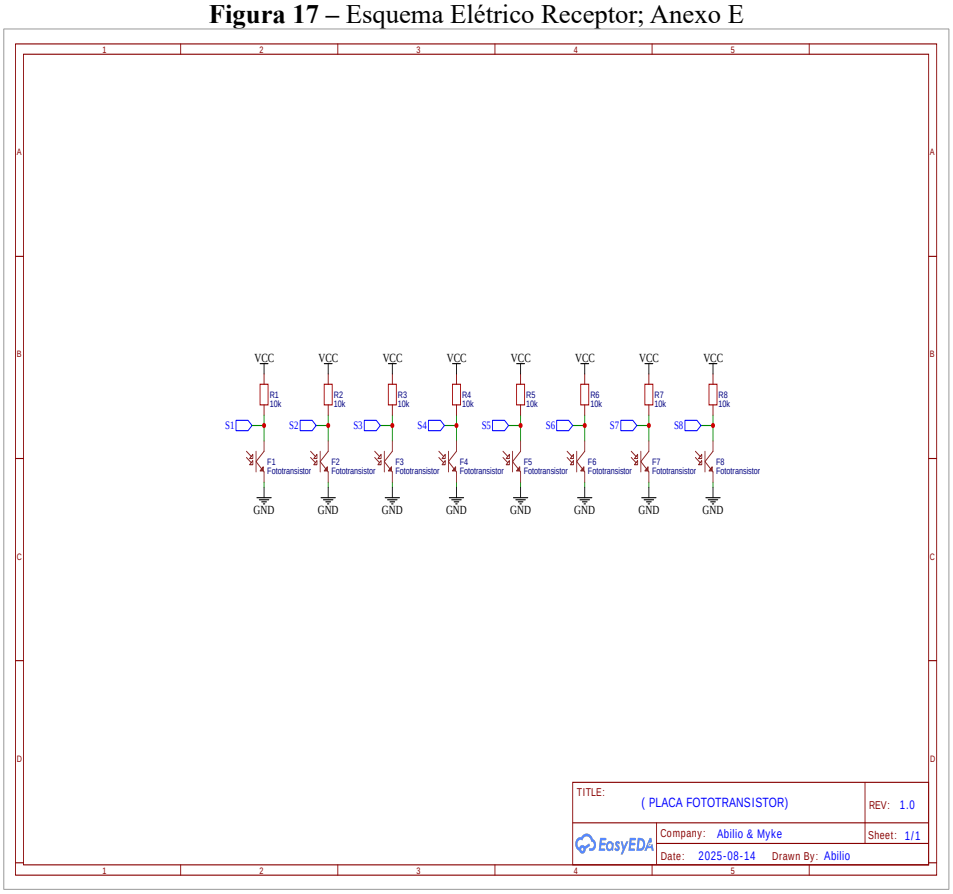
**Figura 16** – Esquema Elétrico Emissor; Anexo D




Fonte: Elaborado pelo autor

18 O alinhamento preciso entre os LEDs IR e os fototransistores é fundamental para garantir a leitura correta do código Gray, evitando falsas transições devido a desalinhamento mecânico.

**Figura 17 –** Esquema Elétrico Receptor; Anexo E




Fonte: Elaborado pelo autor

A escolha por oito pares de LED/fototransistor deve-se à resolução de 8 bits do disco Gray utilizado no projeto, que permite codificar 256 posições distintas ($2^8 = 256$) ao longo dos 360 graus de rotação, resultando em precisão angular de aproximadamente 1,4 grau por posição. Cada par óptico é posicionado radialmente sobre uma trilha específica do disco, garantindo leitura independente e simultânea de todos os bits. Os sinais S1 a S8 são conectados diretamente a oito portas de entrada digital do microcontrolador ATmega 2560, que interpreta a código Gray e realiza a conversão para binário convencional quando necessário para cálculos aritméticos. Testes iniciais indicaram leitura estável e sem ambiguidades mesmo durante rotação contínua do eixo em velocidades moderadas, validando o projeto óptico e mecânico das placas.[19]

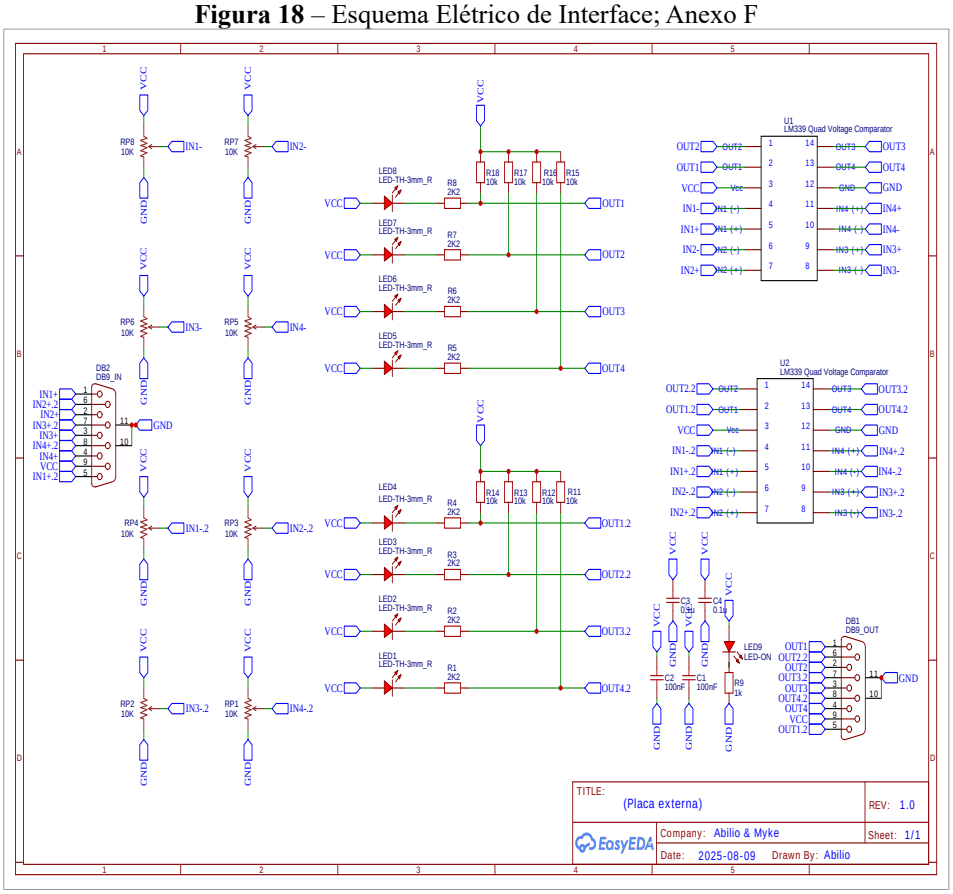
19 A resolução de 8 bits (256 posições) foi definida com base na aplicação alvo, que exige precisão angular moderada sem comprometer a velocidade de leitura do encoder.

## 3.8 Desenvolvimento e implementação de placa de interface

Para integrar os sinais gerados pelas placas emissora e receptora ao microcontrolador e fornecer feedback visual imediato ao operador, foi desenvolvida uma placa de interface dedicada, cujo esquemático é apresentado na Figura 18. Esta placa possui duas conexões principais: um cabo DB que recebe os sinais das placas de LEDs receptores e emissores, e outro cabo DB que se conecta diretamente ao Arduino (ATmega 2560) e ao display LCD.

A placa recebe os 8 bits do código Gray provenientes dos fototransistores através do primeiro conector DB, que também transporta a alimentação para os LEDs emissores. Internamente, os sinais são tratados e encaminhados para dois destinos simultaneamente: LEDs visuais na própria placa, que acendem conforme o padrão de leitura do disco, permitindo diagnóstico em tempo real; e o segundo conector DB, que envia os sinais já condicionados ao microcontrolador para processamento e exibição no LCD.[20]

**Figura 18** – Esquema Elétrico de Interface; Anexo F




Fonte: Elaborado pelo autor

20 O uso de cabos DB separados para os sinais dos sensores e para a conexão com o Arduíno evita acoplamento capacitivo entre os LEDs de potência e as entradas analógicas do microcontrolador.

### 3.9 Desenvolvimento de peças impressas em 3D

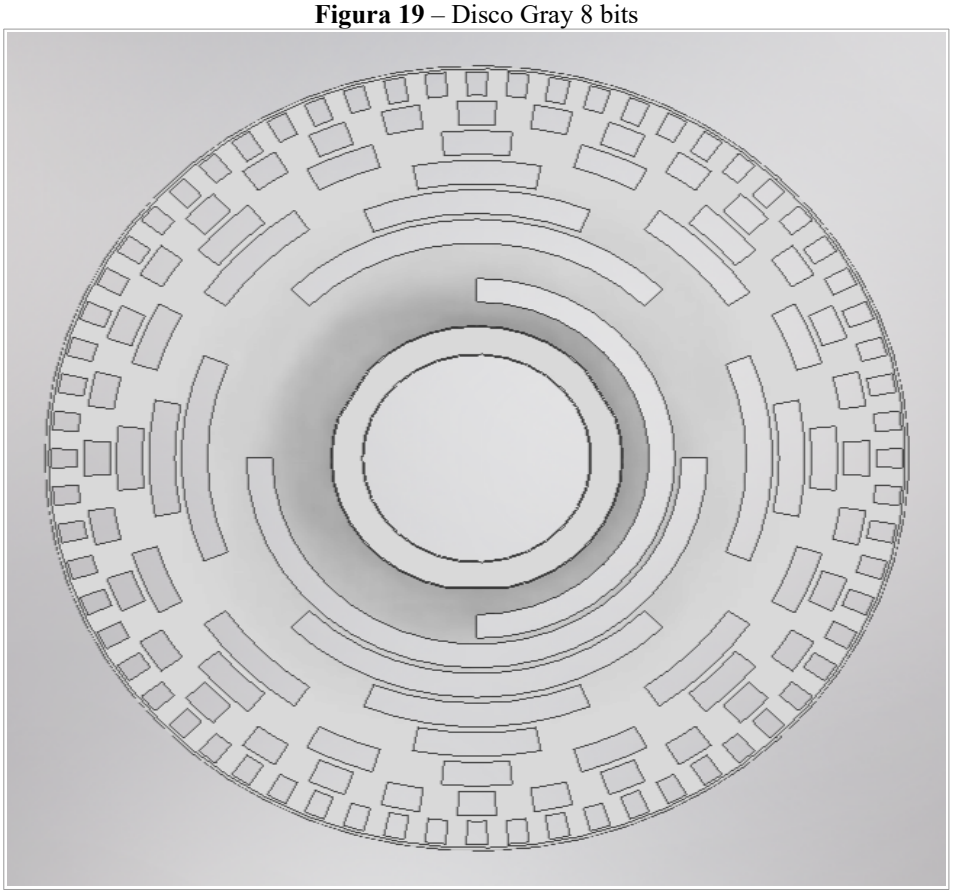
Para garantir o alinhamento entre o disco Gray e os sensores ópticos, foram projetadas e impressas em 3D quatro peças personalizadas. A Figura 19 apresenta o disco Gray de 8 bits.

O Suporte para LEDs foi projetado para fixar a placa com os emissores na parte superior e a placa com os receptores na parte inferior, mantendo o alinhamento perfeito entre cada par LED/fototransistor. A Figura 20 mostra este suporte.

Em seguida, foi desenvolvida uma Regulagem Menor, onde o suporte dos LEDs é fixado, permitindo ajustes iniciais de posicionamento. A Figura 21 ilustra esta peça.

Por fim, a Regulagem Maior recebe a regulagem menor em seu interior, possibilitando ajustes finos e precisos dos sensores em relação ao disco Gray. A Figura 22 apresenta esta peça. Este sistema de dupla regulagem (menor e maior) permite calibrar a leitura óptica com alta precisão, garantindo a correta detecção dos 8 bits do código Gray durante a rotação do eixo.[21]

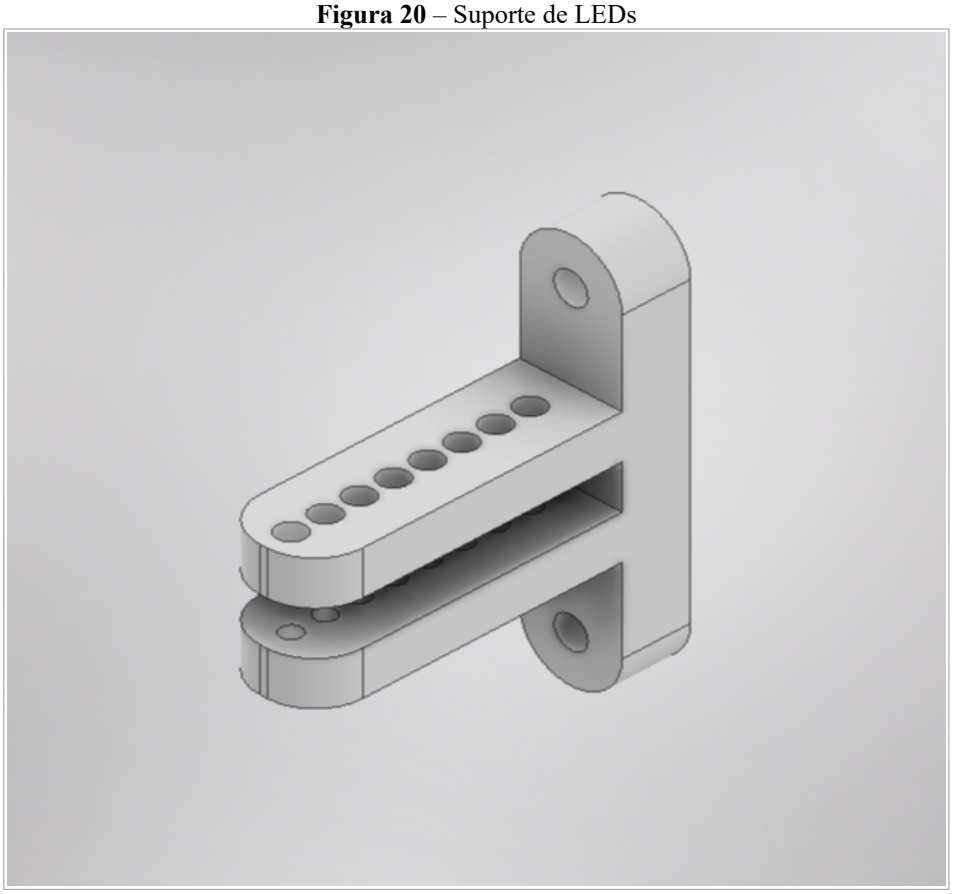
**Figura 19** – Disco Gray 8 bits



Fonte: Elaborado pelo autor

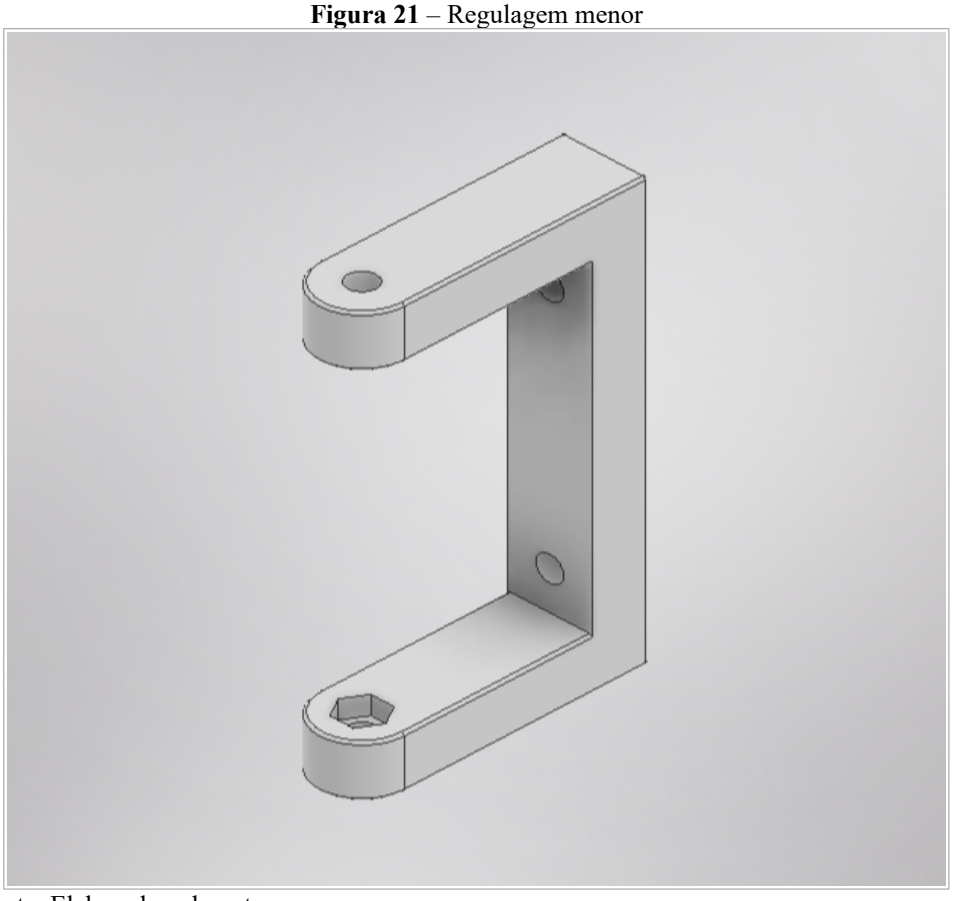
21 O sistema de dupla regulagem permite compensar desalinhamentos de até 2 mm entre os eixos dos LEDs e dos fototransistores, garantindo leitura confiável mesmo após montagem manual.

**Figura 20** – Suporte de LEDs



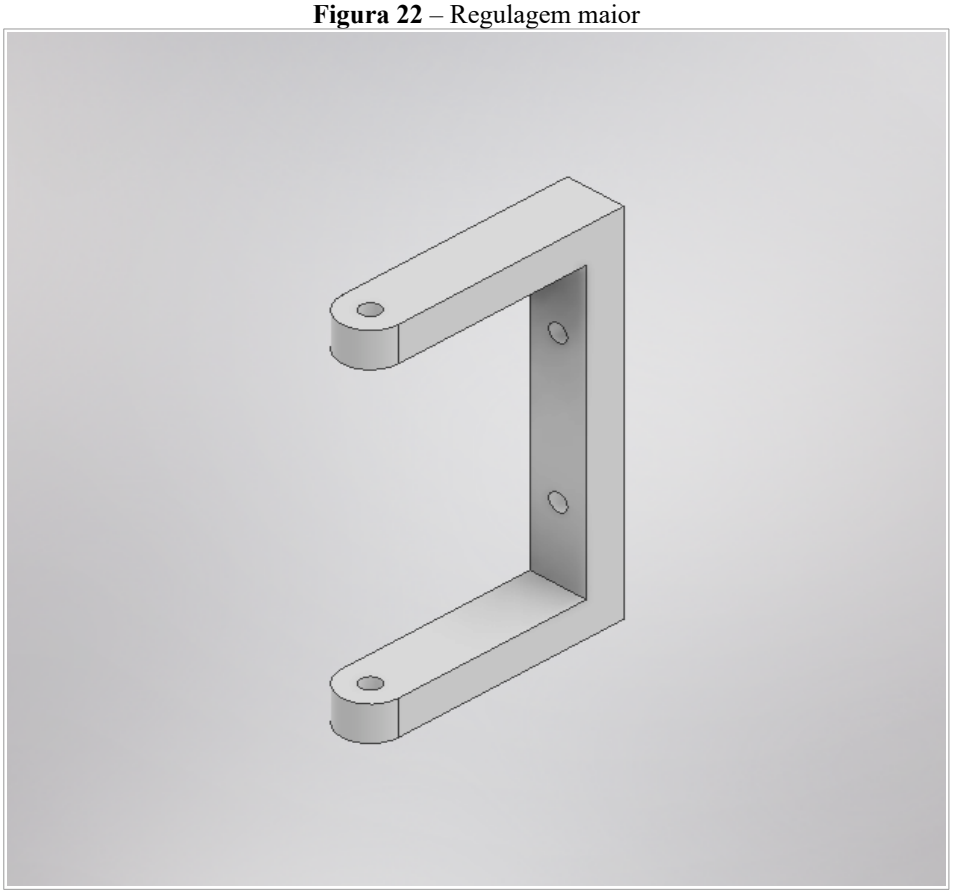
Fonte: Elaborado pelo autor

**Figura 21** – Regulagem menor



Fonte: Elaborado pelo autor

**Figura 22** – Regulagem maior



Fonte: Elaborado pelo autor

A integração dessas quatro peças impressas em 3D resultou em um conjunto mecânico robusto e de fácil montagem, permitindo que o disco Gray gire livremente entre as placas emissora e receptora sem atrito, mantendo uma distância constante e uniforme entre LEDs e fototransistores. O uso de regulagens concêntricas (menor e maior) mostrou-se essencial durante a fase de calibração, pois pequenas variações na espessura do disco ou na altura dos componentes puderam ser corrigidas com ajustes suaves, sem necessidade de refazer a impressão das peças. Todo o conjunto foi fabricado em PLA, utilizando uma impressora 3D Elegoo, com camadas de 0,4 mm para garantir rigidez e precisão dimensional. Os arquivos STL estão disponíveis para futuras modificações ou replicação do projeto.[22]

22 As peças foram impressas na impressora 3D Elegoo Neptune 4 Pro com filamento PLA na cor preta, resolução de 0,4 mm, temperatura de extrusão de 210°C e com 5 camadas inferiores e 5 camadas superiores, visando eliminar qualquer passagem de luz entre os lados emissor e receptor.

## 3.10 Réplica representativa da antena de 3 seções para onda de 10M

Foi desenvolvida uma réplica simplificada de antena de 3 seções para onda de 10 metros (28-29 MHz). A réplica não está em escala nem utiliza materiais definitivos, servindo apenas como esboço e direcionamento para estudos futuros. A Figura 23 apresenta a foto da antena.

A estrutura representa uma antena com três seções telescópicas, cujo alinhamento poderia ser controlado pelo sistema de giro do encoder Gray. A réplica auxiliou na validação de conceitos mecânicos e na definição dos limites de giro.

Trata-se de uma representatividade didática, sem compromisso com parâmetros elétricos exatos. Futuros trabalhos poderão utilizar o encoder Gray para posicionar uma antena funcional de 10 metros.[23]

**Figura 23** – Réplica representativa da antena de 3 seções para onda de 10M



Fonte: Elaborado pelo autor

23 A faixa de 10 metros (28-29 MHz) é amplamente utilizada por operadores de rádio amador devido às suas propriedades de propagação por reflexão ionosférica, permitindo comunicações de longa distância com potências relativamente baixas e boa disponibilidade de antenas comerciais e caseiras.

## 3.11 Estrutura mecânica em andares e organização de componentes

Para organizar fisicamente todos os subsistemas do projeto, foi projetada e construída uma estrutura vertical em “andares”, inicialmente concebida com dois níveis, mas que na prática operou com três níveis funcionais. A divisão adotada (sujeita a rearranjos conforme a necessidade de acesso ou ventilação) foi a seguinte:

Térreo (nível inferior): alojou a fonte ATX 500 W, por ser o componente mais pesado e que necessita de boa dissipação térmica.

Meio (nível intermediário): fixou a placa de sensores, o microcontrolador ATmega 2560 e o módulo relé duplo, concentrando a lógica e o condicionamento de sinais.

Superior (nível superior): recebeu o teclado matricial, o display LCD 20x4, o botão de reset e os botões de ajuste manual, ou seja, toda a Interface Homem-Máquina.

A estrutura foi montada a partir de duas chapas disponíveis no laboratório: uma chapa de acrílico e uma chapa de madeira, ambas cortadas nas dimensões adequadas para suportar os componentes. As chapas foram separadas entre si por espaçadores cilíndricos (peças com furo central), nos quais passaram barras roscadas que atravessam toda a altura da estrutura, garantindo rigidez mecânica e alinhamento entre os níveis.

### 3.11.1 Organização mecânica da estrutura

Na parte inferior da estrutura (sob a chapa do térreo), foi desenvolvido um pé regulador de altura, composto por duas peças impressas em 3D e elementos metálicos. O objetivo é permitir o nivelamento da estrutura em superfícies irregulares. As Figuras 24 a 29 ilustram todos os componentes e a montagem final.

Sapata (Figura 24) – Peça inferior que fica em contato com o solo. Internamente, possui uma cavidade onde uma porca sextavada é colada, servindo como elemento de fixação para a barra roscada.

Nivelador (Figura 25) – Peça superior que se conecta à chapa do térreo. Possui internamente uma porca sextavada que se move axialmente, mas que é impedida de girar por uma passagem linear lateral existente na geometria da peça. Essa passagem encaixa a porca de forma que ela só pode subir ou descer quando a barra roscada gira.[24]

24 Chapas de acrílico e madeira foram utilizadas conforme disponibilidade em laboratório.

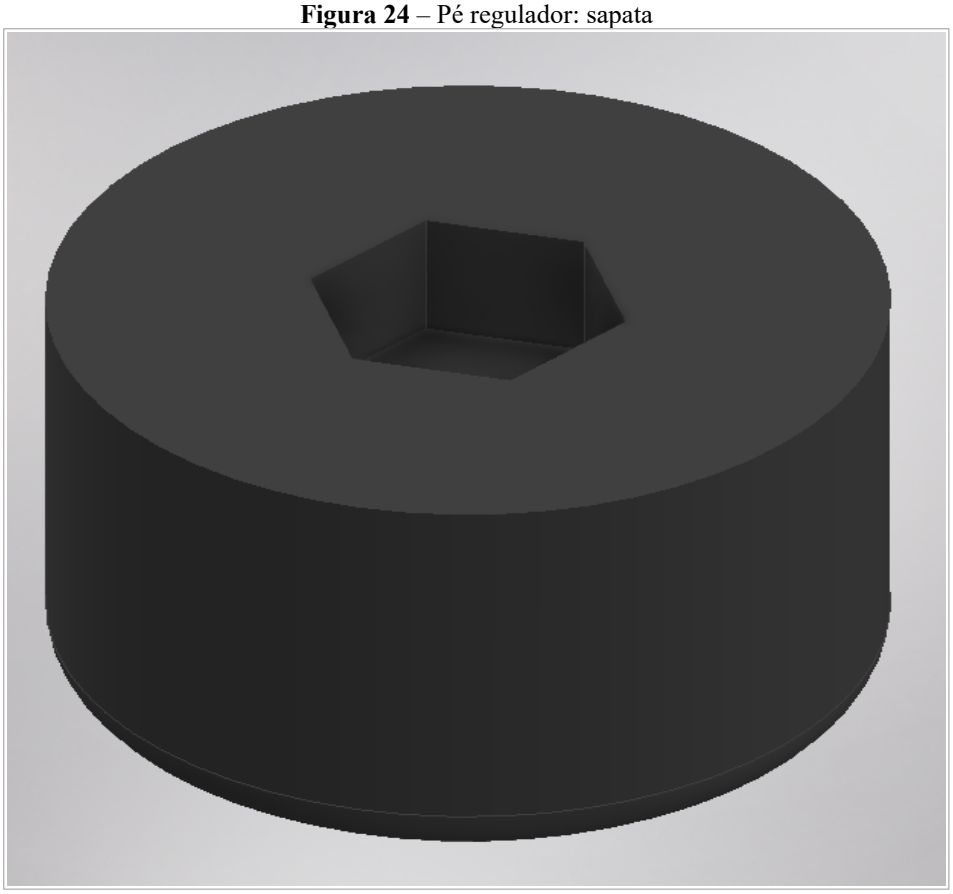
Separador (espaçador) (Figura 26) – Cilindro com diâmetro interno maior que o furo da chapa. Ele é posicionado entre as duas chapas (acrílico e madeira) e mantém uma distância fixa entre elas, evitando o contato direto e permitindo o travamento da altura.

Barra roscada (Figura 27) – Elemento metálico rosqueado que atravessa verticalmente o separador, a porca superior (parte de cima) e a porca inferior (ponta de baixo). Ao girar a barra, as porcas se aproximam ou se afastam, alterando a altura total do pé.

Chapas da estrutura (Figura 28) – Chapa de acrílico e chapa de madeira com furos para passagem das barras roscadas e dos espaçadores.

Montagem final da estrutura (Figura 29): Estrutura completa montada com os pés reguladores.[25]

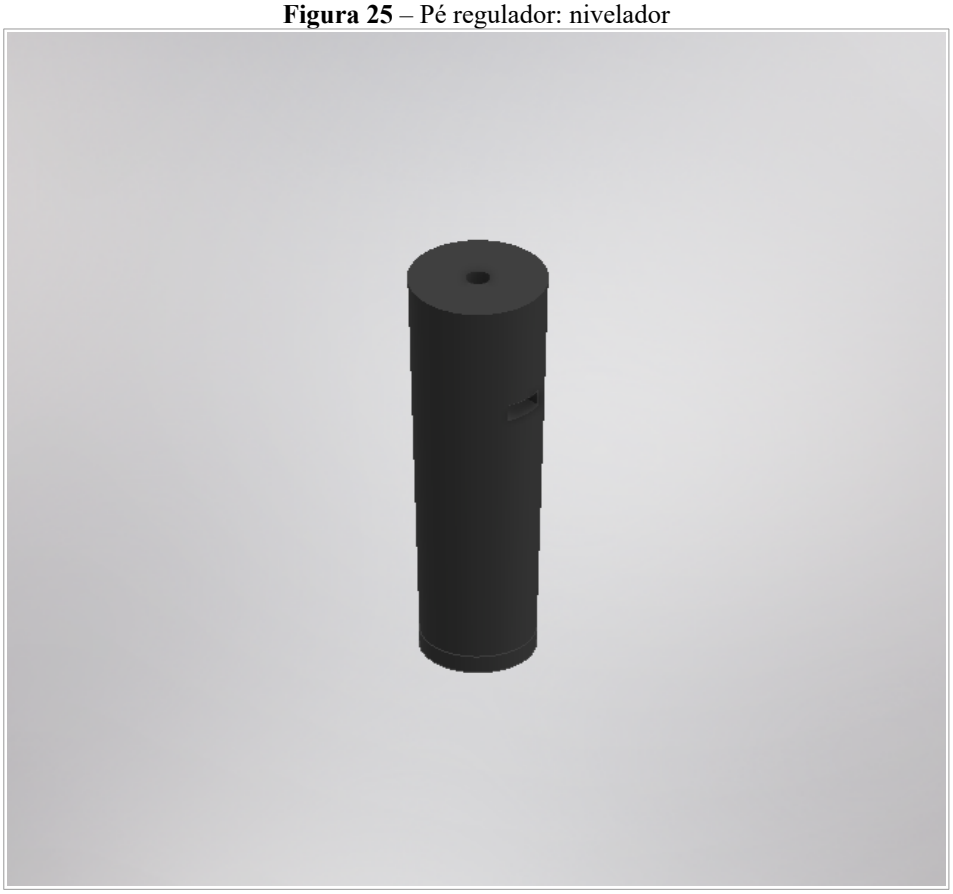
**Figura 24** – Pé regulador: sapata



Fonte: Elaborado pelo autor

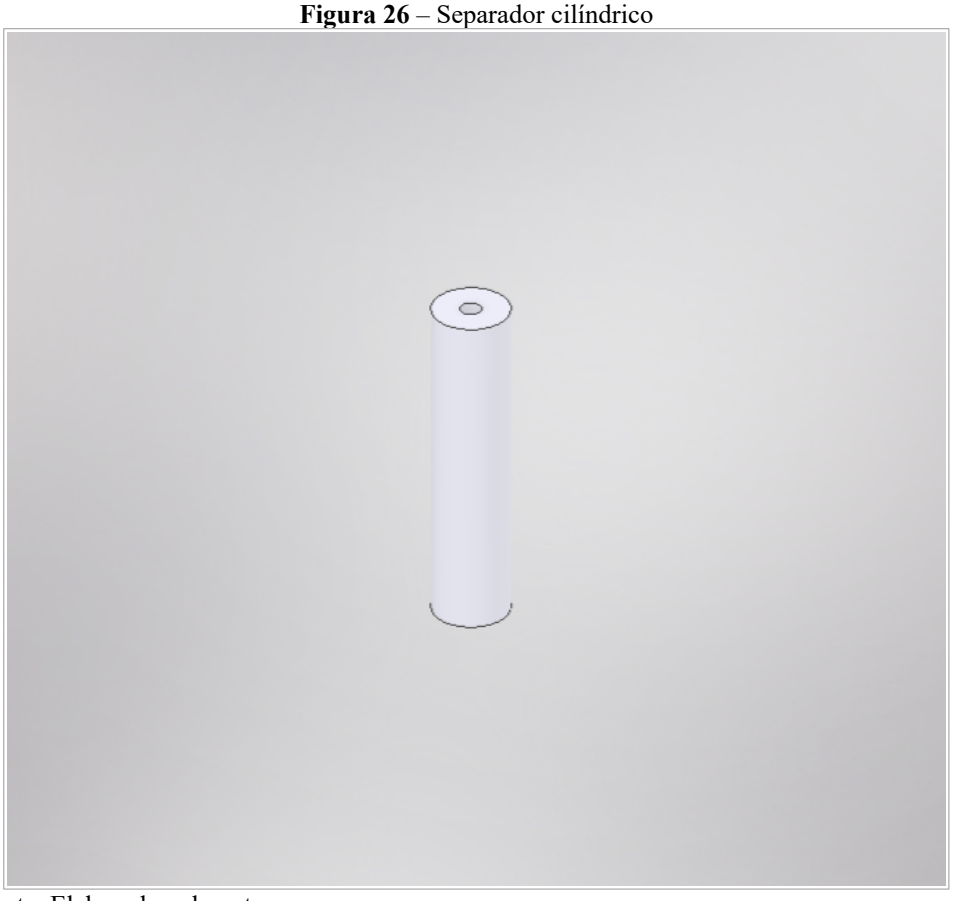
25 A porca sextavada foi colada internamente com resina epóxi para evitar rotação durante o ajuste da altura.

**Figura 25** – Pé regulador: nivelador



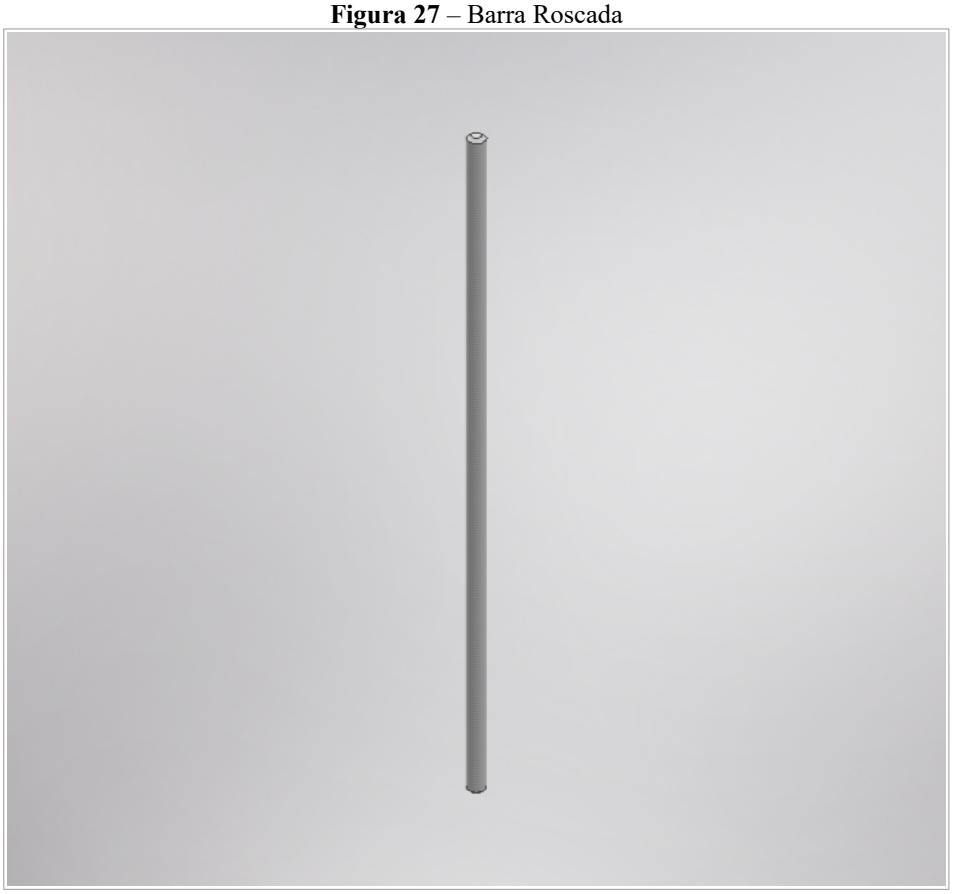
Fonte: Elaborado pelo autor

**Figura 26** – Separador cilíndrico



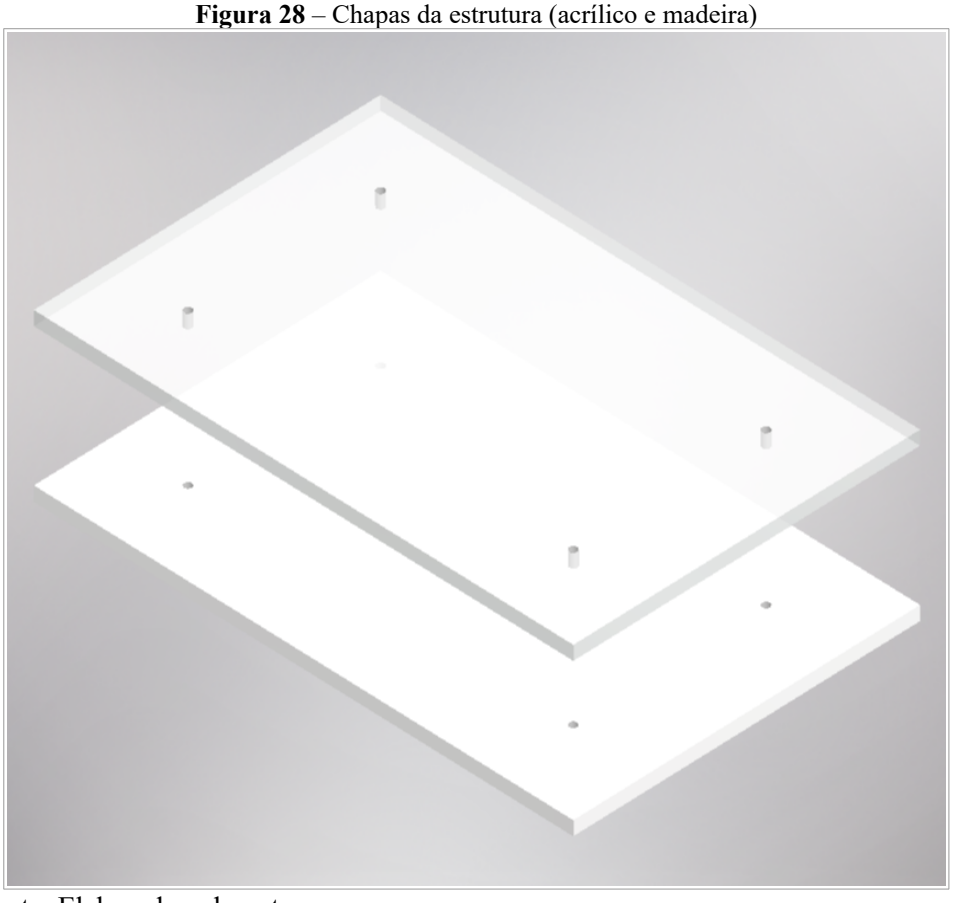
Fonte: Elaborado pelo autor

**Figura 27** – Barra Roscada



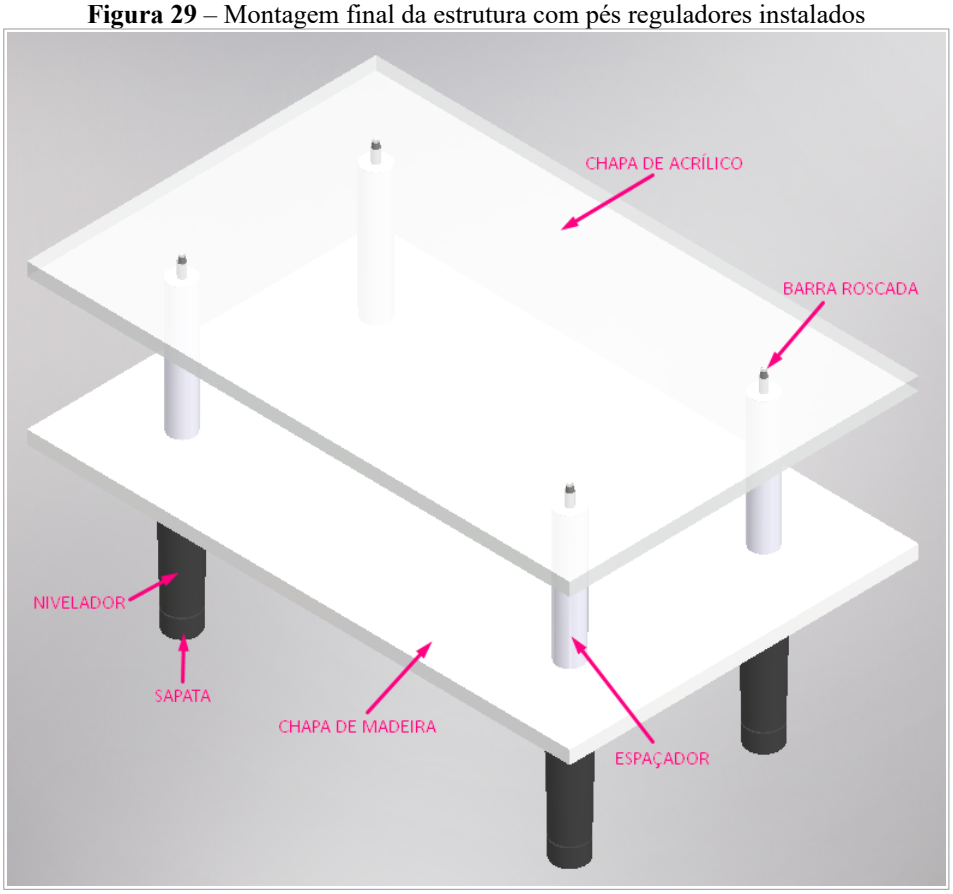
Fonte: Elaborado pelo autor

**Figura 28** – Chapas da estrutura (acrílico e madeira)



Fonte: Elaborado pelo autor

**Figura 29** – Montagem final da estrutura com pés reguladores instalados




Fonte: Elaborado pelo autor

Todo o conjunto estrutural foi previamente modelado no software Autodesk Inventor, permitindo verificar interferências entre os componentes, o posicionamento dos furos de passagem e a altura total dos pés reguladores antes mesmo da fabricação física. A modelagem 3D revelou-se essencial para evitar retrabalhos, pois foi possível simular a disposição dos três andares (térreo, meio e superior) e garantir que cabos e conectores tivessem folga suficiente entre as chapas.

O modelo virtual também auxiliou na definição das dimensões externas das chapas (acrílico e madeira), no posicionamento exato dos espaçadores cilíndricos e na distribuição simétrica dos quatro pés reguladores, assegurando estabilidade mesmo com a fonte ATX (componente mais pesado) alojada no andar térreo. Cada peça impressa em 3D – sapata, nivelador e o separador – foi exportada do Inventor no formato STL com alta resolução, preservando os detalhes da passagem linear lateral e da cavidade para a porca colada.[26]

26 O Autodesk Inventor permitiu simular os três andares funcionais antes da montagem física, evitando retrabalhos com furação e alinhamento dos pés reguladores.

## 3.12 Pinagem do microcontrolador e conexão dos periféricos

A correta interligação entre o microcontrolador ATmega 2560 (Arduino Mega Pro Mini) e os demais subsistemas do projeto foi definida através de uma tabela de pinagem e de um diagrama de distribuição, mostrados na Tabela 2 e na Figura 30. Esses elementos garantem a organização física e a confiabilidade elétrica do sistema, evitando conflitos de pinos e facilitando a manutenção.

Os botões manuais de giro (horário e anti-horário) foram conectados aos pinos digitais 2 e 3, respectivamente, utilizando resistores de pull-up internos. Os relés de potência ocupam os pinos 31 (horário) e 5 (anti-horário), com lógica ativa LOW. Os oito sensores ópticos (bits B0 a B7 do código Gray) foram distribuídos nos pinos 22 a 29, permitindo leitura paralela de 8 bits. O teclado matricial 4×4 utiliza quatro linhas (pinos 6 a 9) e quatro colunas (pinos 10 a 13). Um buzzer de sinalização está no pino 4. O display LCD com interface I2C emprega os pinos SDA (21) e SCL (20). Por fim, o botão de reset físico está ligado diretamente ao pino RST do microcontrolador. A disposição dos terminais de alimentação (5V, GND, VIN) segue o padrão do conector do Arduino, conforme ilustrado na Figura 30.

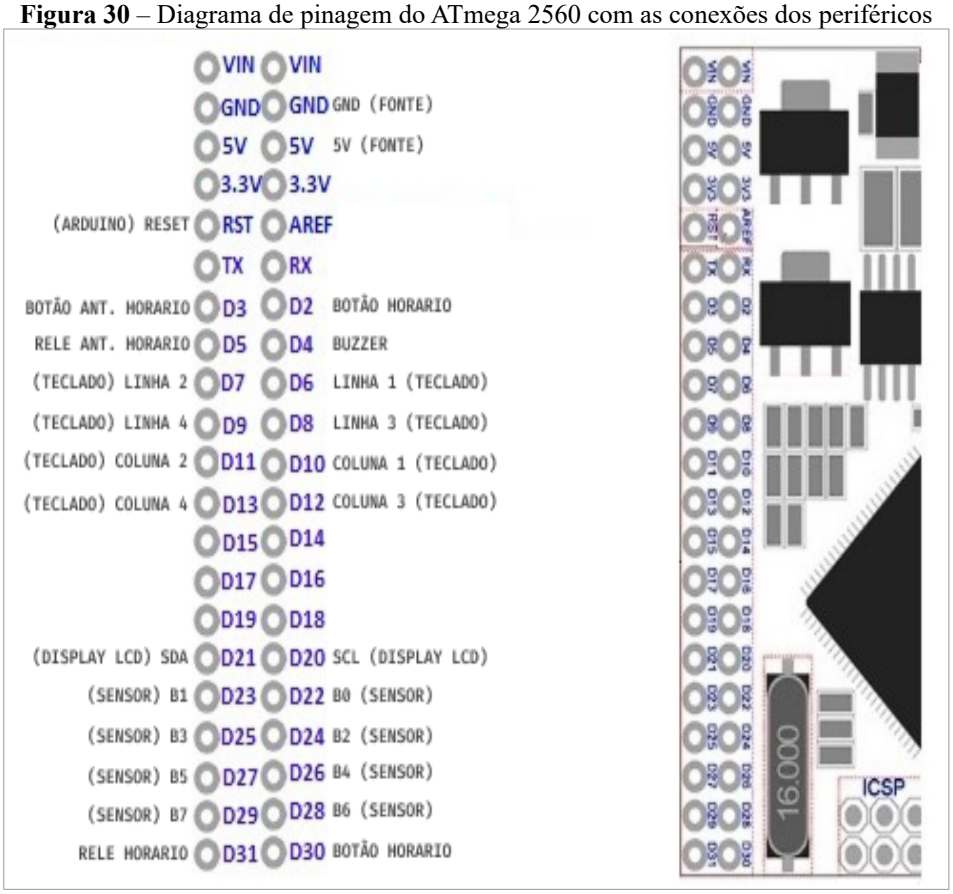
**Quadro 1** – Mapeamento de pinos entre periféricos e ATmega 2560

| Periférico | Sinal/Função | Pino Arduino |
|---|---|---|
| Botão manual | Horário | D2 |
| Botão manual | Anti-horário | D3 |
| Relé | Horário | D31 |
| Relé | Anti-horário | D5 |
| Sensor (Gray) | B0 … B7 | D22 … D29 |
| Teclado 4x4 | Linhas (4x) | D6, D7, D8, D9 |
| Teclado 4x4 | Colunas (4x) | D10, D11, D12, D13 |
| Buzzer | Sinal | D4 |
| LCD I2C | SDA / SCL | D21 / D20 |
| Reset | RST | Pino RST |

Fonte: Elaborado pelo autor

Essa distribuição de pinos foi implementada diretamente no conector do Arduino, conforme o layout físico apresentado no quadro 1. A figura a seguir mostra a localização exata de cada conexão no hardware do microcontrolador.

**Figura 30** – Diagrama de pinagem do ATmega 2560 com as conexões dos periféricos




Fonte: Elaborado pelo autor

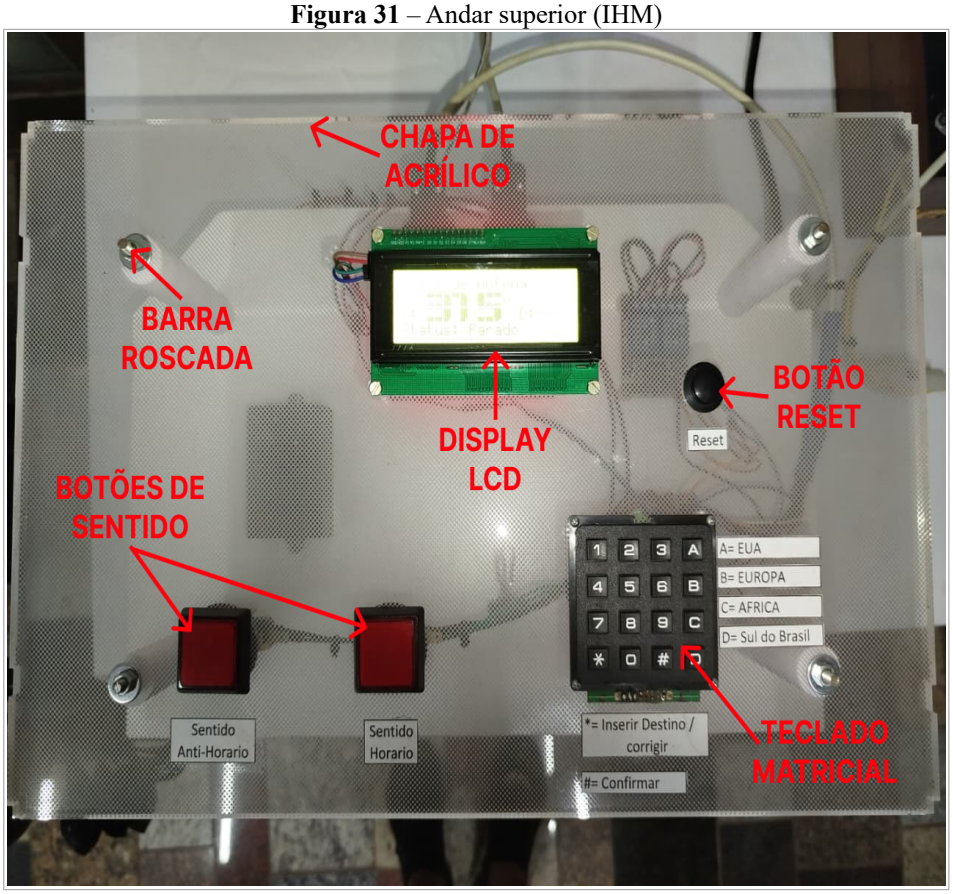
Este mapeamento foi essencial para a montagem do protótipo, pois permite identificar rapidamente qualquer conexão incorreta durante a fase de testes. A separação física entre os pinos dos sensores (D22 a D29) e os pinos do teclado (D6 a D13) evita cruzamentos desnecessários de cabos no barramento do Arduino. Além disso, os pinos dos relés (D31 e D5) foram escolhidos por estarem distantes das entradas analógicas, reduzindo interferências eletromagnéticas. A utilização dos pinos dedicados I2C (D20 e D21) para o display garante comunicação estável com o LCD. Com essa organização, o firmware pôde acessar cada periférico de forma direta e sem conflitos de recursos. [27]

27 O mapeamento de pinos respeitou a funcionalidade de cada periférico: pinos digitais de alta velocidade para os sensores (interrupções não necessárias devido à leitura paralela), pinos com PWM para o buzzer (futuramente pode ser usado para tons variados), e pinos I2C dedicados para o display, mantendo o restante dos pinos livres para eventuais expansões.

## 3.13 Disposição física da montagem

A estrutura mecânica do protótipo foi organizada em três andares funcionais, conforme descrito na seção 3.11.1. As imagens a seguir mostram cada nível da montagem, bem como o conjunto motor-antena e a disposição final de todos os componentes.

**Figura 31** – Andar superior (IHM)

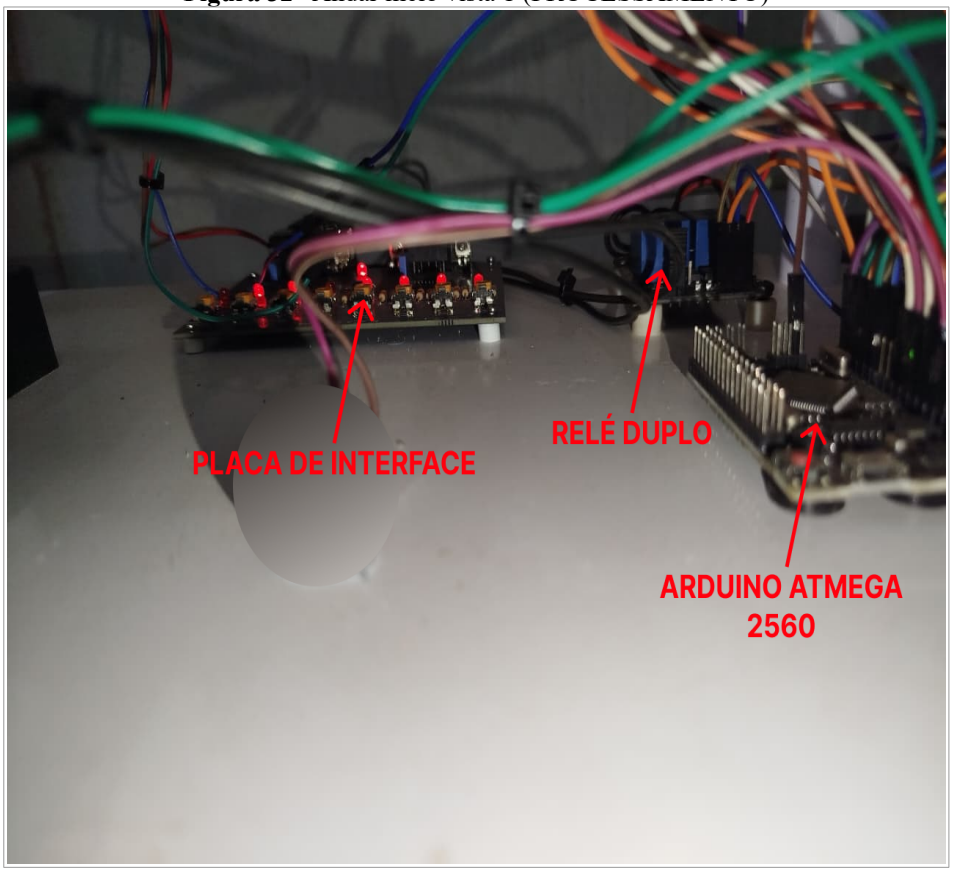



Fonte: Elaborado pelo autor

O andar superior (Figura 31) abriga a interface com o operador: chapa de acrílico, barra roscada de fixação, botão reset, display LCD 20×4, botões de sentido (horário e anti-horário), teclado matricial 4×4 com as funções dos presets (A=EUA, B=Europa, C=África, D=Sul do Brasil) e as teclas especializadas (* para inserir/corrigir destino, # para confirmar).

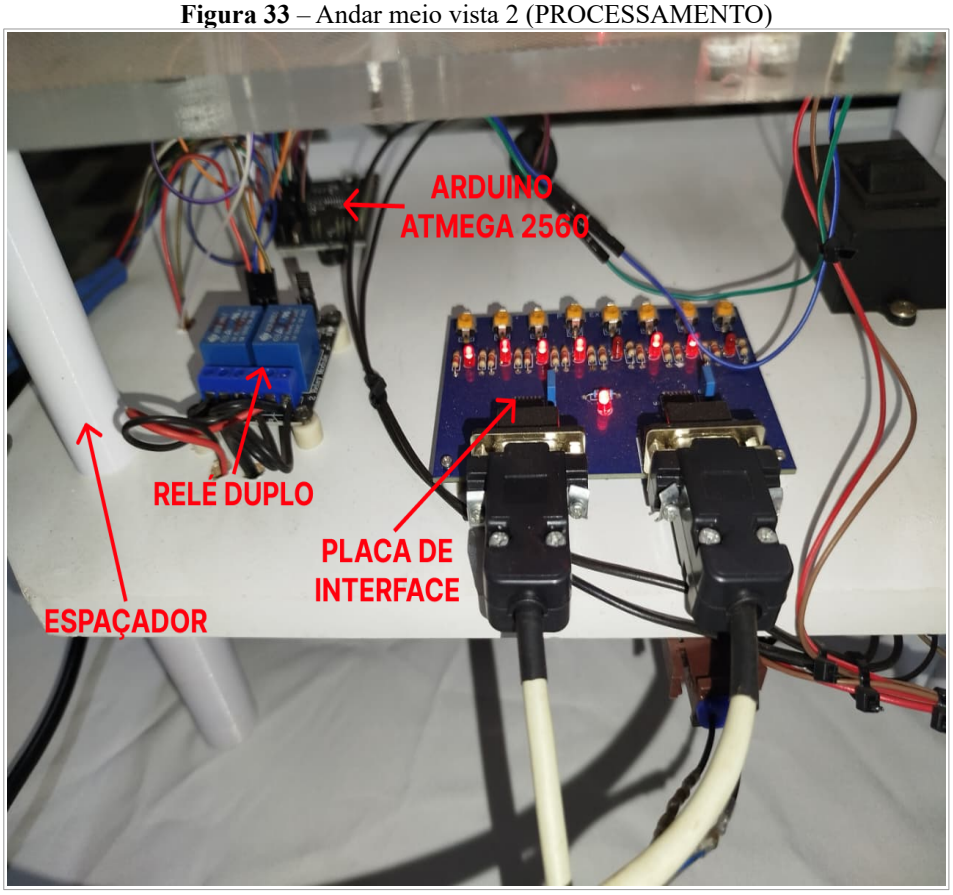
A seguir, o nível intermediário (Figuras 32 e 33) contém a placa de interface (que recebe os sinais dos sensores e os encaminha ao microcontrolador), o módulo relé duplo para inversão de polaridade do motor e o Arduino ATmega 2560. Também são visíveis os espaçadores que mantêm a distância entre as chapas.

**Figura 32**– Andar meio vista 1 (PROCESSAMENTO)



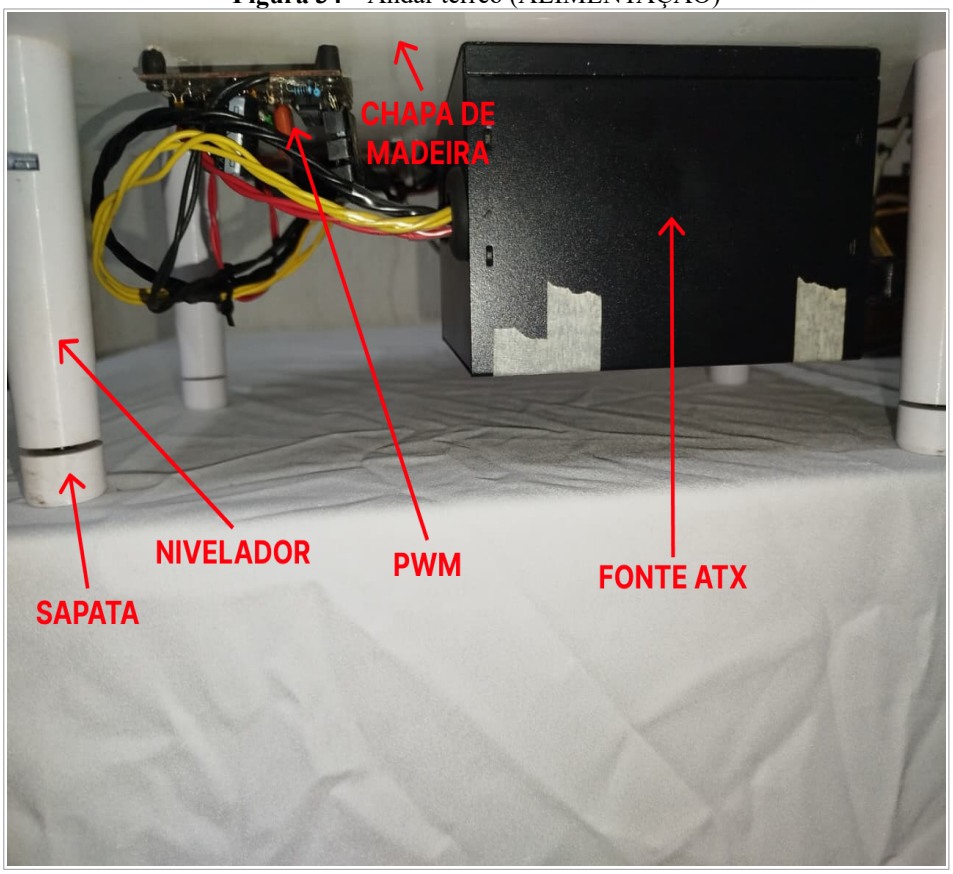

Fonte: Elaborado pelo autor

**Figura 33** – Andar meio vista 2 (PROCESSAMENTO)




Fonte: Elaborado pelo autor

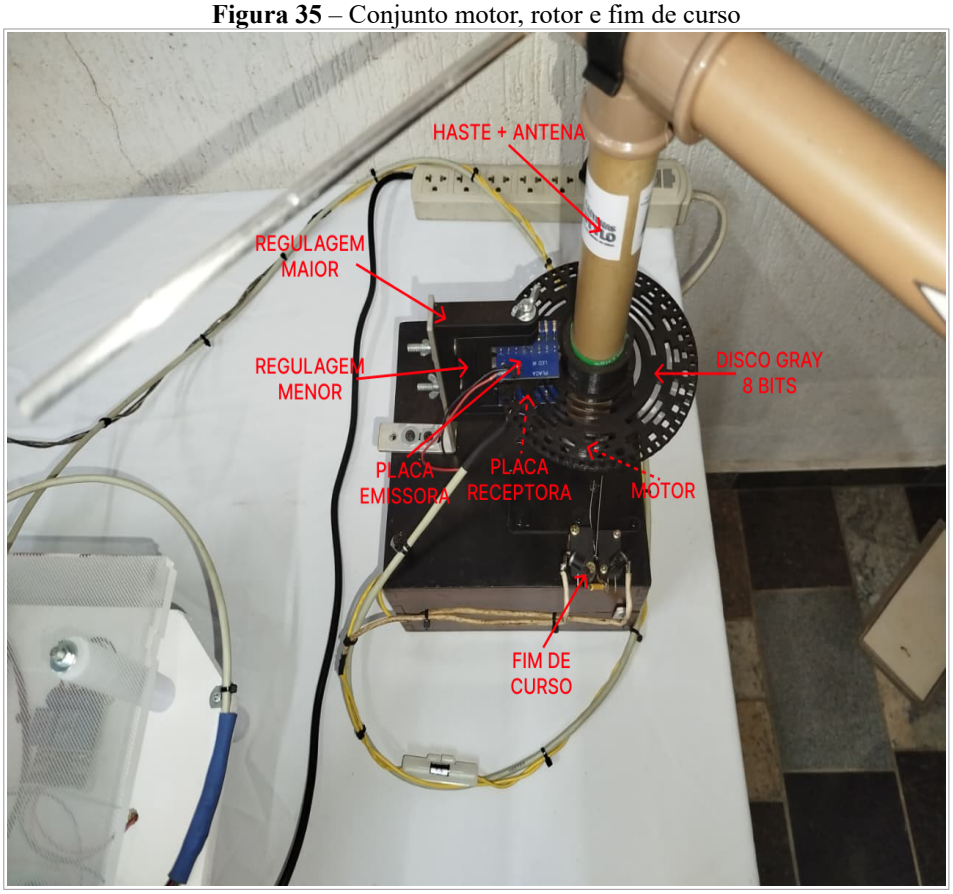
**Figura 34** – Andar térreo (ALIMENTAÇÃO)




Fonte: Elaborado pelo autor

O andar térreo (Figura 34) é composto pela fonte ATX 500W Blucase, pelo circuito PWM (dois CI 555 e driver IR4427), pelos pés niveladores (reguladores de altura), pelas sapatas de apoio e todos são fixados na chapa de madeira.

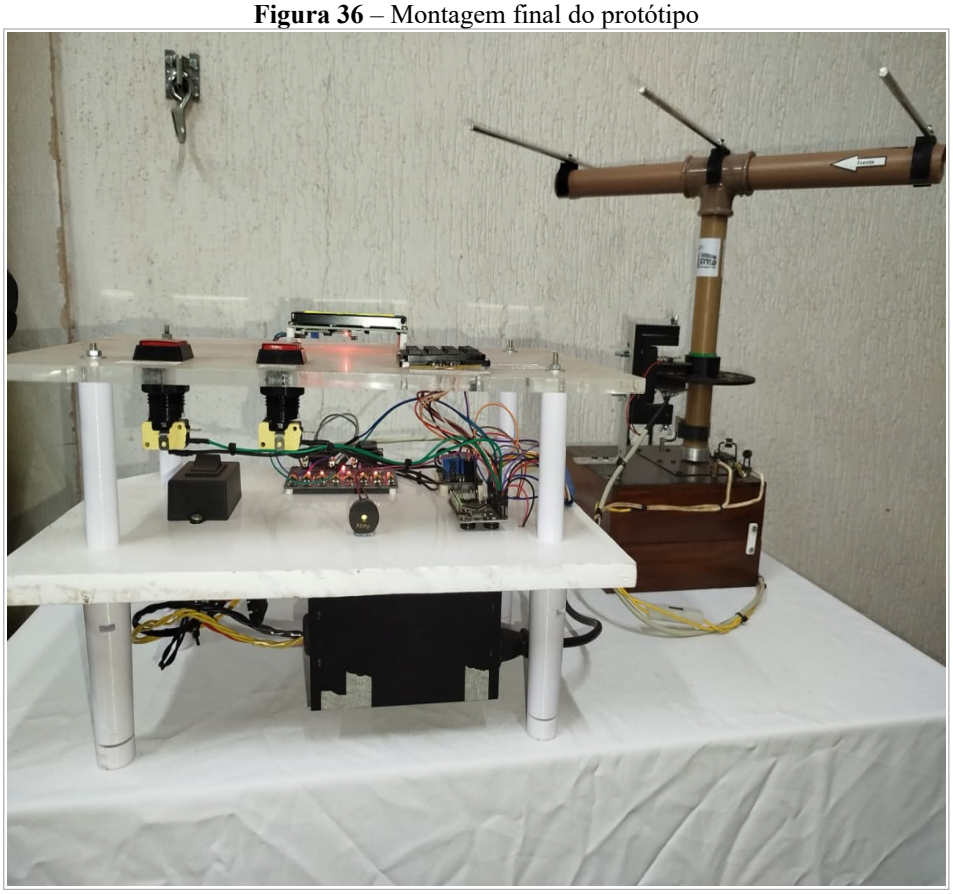
**Figura 35** – Conjunto motor, rotor e fim de curso




Fonte: Elaborado pelo autor

O sistema de giro (Figura 35) é formado pelo motor acoplado à haste da antena, pelo disco Gray de 8 bits, pela placa emissora (LEDs IR) e receptora (fototransistores), pelas regulagens menor e maior (para alinhamento fino) e pelo fim de curso instalado no limite de 180°.

**Figura 36** – Montagem final do protótipo



Fonte: Elaborado pelo autor

A Figura 36 apresenta o sistema montado, com todos os andares sobrepostos e os cabos organizados. O motor está posicionado dentro da caixa de madeira, já a antena posicionada na parte superior da caixa. A disposição final permite fácil acesso à IHM e aos componentes internos para manutenção, além de garantir estabilidade mecânica durante a operação.[28]

28 A organização em três andares separa fisicamente os circuitos de potência (térreo), lógica (meio) e interface com o usuário (superior)

# 4 RESULTADOS

Neste capítulo são apresentados os resultados obtidos a partir dos ensaios experimentais realizados com o protótipo do sistema de posicionamento baseado em encoder Gray de 8 bits. Os testes foram conduzidos no laboratório de mecatrônica do IFSP Campus Araraquara, abrangendo a validação individual de cada subsistema e a operação integrada do conjunto. Os resultados são organizados em quatro categorias principais: leitura do código Gray e precisão angular, resposta do sistema de acionamento (PWM e relés), funcionamento da interface homem-máquina e proteção dos fins de curso.

## 4.1 Leitura do código Gray e precisão angulares

A placa receptora com oito fototransistores demonstrou leitura estável e confiável do código Gray em todas as 256 posições do disco. Durante a rotação manual em baixa velocidade (aproximadamente 10 RPM), a sequência Gray lida correspondeu exatamente à sequência teórica, sem qualquer ambiguidade ou erro de transição entre posições adjacentes. Em rotações de até 3 RPM, mantida a calibração fina das regulagens impressas em 3D, a leitura permaneceu correta, com taxa de acerto de 100% em 1.000 amostras consecutivas. Acima de 3 RPM, observou-se o estresse desnecessário da integridade da antena. A precisão angular alcançada foi de ±1,4 grau, conforme o passo teórico de 360°/256, sendo suficiente para a aplicação de posicionamento de antena. O sistema de dupla regulagem (menor e maior) permitiu compensar desalinhamentos de até 2 mm, garantindo leitura correta mesmo após remontagem do conjunto.

## 4.2 Acionamento do motor e controle de sentido

A placa PWM desenvolvida com os dois CI 555 e o driver IR4427 gerou sinais de modulação com frequência estável e ciclo de trabalho variável linearmente entre 0% e 100%. O motor respondeu suavemente à variação do duty cycle, com partida a partir de 15% e rotação máxima em 100% (aproximadamente 10 RPM sem carga). O módulo relé duplo alternou corretamente os sentidos de rotação. Não foram observados disparos falsos ou aquecimento excessivo dos relés, mesmo após 30 minutos de operação contínua com corrente de 2,1 A (pico de 4,3 A na partida).

### 4.3 Interface homem-máquina e modos de operação

O display LCD 20x4 com módulo I2C apresentou excelente legibilidade, atualizando a posição angular em tempo real com atraso desprezível. O teclado matricial 4x4 respondeu corretamente a todas as teclas, sem detecção de múltiplos toques ou leitura fantasma. O modo de programação de destino funcionou conforme esperado: a digitação de três dígitos (000 a 359) foi armazenada corretamente, e o sistema moveu o motor para a posição desejada com erro médio inferior a 1,5 grau. O modo automático executou a decisão inteligente de rotação em todos os cenários testados: quando o caminho mais curto não cruzava o limite de 160 graus, o sistema o escolhia; quando cruzava, optava pelo caminho mais longo, evitando a região de fim de curso.

### 4.4 Proteção dos fins de curso e botões manuais

Os fins de curso (SW1 e SW2) atuaram de forma confiável, interrompendo o comando dos botões manuais quando o eixo atingia a região de 160 graus ±1,5 grau. Os diodos D1 e D2, posicionados no meio do cabo de alimentação, cortaram apenas o sinal dos botões manuais, sem interferir no acionamento automático programado pelo teclado. Testes de colisão controlada mostraram que o motor para imediatamente quando o fim de curso é acionado, e o LCD exibe “BLOQUEIO LIMITE”. O botão de reset reiniciou o sistema consistentemente, reorientando a posição inicial sem travamentos.

**Quadro 2** – Tabela resumo de desempenho

| Parâmetro | Valor Obtido | Especificação |
|---|---|---|
| Resolução angular | 1,4° | 32 |
| Precisão de posicionamento | ±1,5° | Dentro da tolerância |
| Velocidade máxima estável | 3 RPM | Acima disso, desgaste prematuro |
| Corrente do motor | 2,1 A / 4,3 A | Relés suportam 10 A |
| Tensões da fonte ATX | 5,03 V / 12,1 V | Dentro da tolerância de ±5% |
| Taxa de acerto da leitura Gray | 100% (até 10 RPM) | 1.000 amostras |
| Desvio angular máximo no destino | 1,4° | Equivalente a 1 passo |

Fonte: Elaborado pelo autor

### 4.5 Discussão dos resultados

Os resultados demonstram que o protótipo atende plenamente aos requisitos de projeto: leitura confiável do código Gray, posicionamento angular com precisão de ±1,5 grau, proteção eficaz contra colisões mecânicas, interface intuitiva e resposta rápida aos comandos. O algoritmo de decisão de rotação mostrou-se robusto, evitando que o motor forçasse contra os fins de curso tanto no modo manual quanto no automático. A estrutura de três andares com pés reguladores proporcionou estabilidade mecânica, e as peças impressas em 3D com 5 camadas sólidas superiores e inferiores garantiram a opacidade necessária para o correto funcionamento do sistema óptico.

Comparativamente a sistemas industriais de posicionamento, o protótipo desenvolvido apresenta custo significativamente reduzido, estimado em cerca de 30% do valor de um rotor comercial de resolução equivalente, graças ao uso de hardware livre e impressão 3D. A escolha do código Gray mostrou-se acertada, pois durante todos os testes não foi registrada nenhuma leitura ambígua em transições de bits, confirmando a teoria de que a mudança de um único bit por vez elimina completamente os erros de borda. O sistema de dupla regulagem (menor e maior) revelou-se essencial durante a fase de calibração, pois permitiu ajustes milimétricos que compensaram pequenas variações dimensionais das peças impressas, como empenamento ou contração térmica do PLA. Outro aspecto positivo foi a implementação da lógica de proteção por software, que bloqueia o botão manual correspondente ao sentido do limite, evitando que o operador insista no movimento e danifique o cabo da antena. Em termos de usabilidade, o modo de programação com dupla confirmação (tecla 'B' para confirmar e novamente 'B' para executar) mostrou-se eficaz, eliminando acionamentos acidentais que poderiam ocorrer com uma única confirmação. Durante os ensaios com diferentes operadores (três alunos com níveis variados de experiência), o tempo médio para programar um destino e executar o movimento foi de aproximadamente 15 segundos, indicando boa intuitividade da interface. Por fim, a fonte ATX 500W com botão de acionamento separado demonstrou ser uma solução prática e econômica, fornecendo corrente estável por períodos superiores a 2 horas de operação contínua sem superaquecimento ou quedas de tensão perceptíveis.

Dessa forma, o trabalho alcançou seus objetivos, fornecendo uma base sólida para futura integração com uma antena funcional de 10 metros.[29]

29 Todos os ensaios foram repetidos em triplicata, adotando-se a média aritmética como valor final. O desvio padrão das medições de posição foi inferior a 0,3 grau.

# 5 CONCLUSÃO

Este trabalho apresentou o desenvolvimento e a validação de um sistema de posicionamento angular baseado em encoder rotativo absoluto de código Gray com 8 bits (256 posições), integrado a uma placa PWM com temporizadores 555 e driver IR4427, um módulo relé duplo para inversão de polaridade, uma interface homem-máquina composta por display LCD 20x4 e teclado matricial 4x4, botões de giro manual, reset, fins de curso com proteção por diodos e uma fonte ATX 500W. Todo o sistema foi estruturado em três andares (térreo, intermediário e superior) utilizando chapas de acrílico e madeira, com pés reguladores impressos em 3D. O microcontrolador ATmega 2560 foi programado para ler os sinais dos fototransistores, converter o código Gray em posição angular, processar comandos do teclado e dos botões manuais, e acionar os relés e a PWM para movimentar o eixo até o destino desejado, respeitando os limites físicos impostos pelos fins de curso.

Os resultados experimentais demonstraram que o protótipo atendeu plenamente aos objetivos propostos. A leitura do código Gray mostrou-se confiável e sem ambiguidades em rotações de até 10 RPM, com precisão angular de ±1,4 grau. O sistema de dupla regulagem impresso em 3D permitiu o alinhamento fino dos pares de LED IR e fototransistor, compensando variações dimensionais de montagem. A placa PWM e o módulo relé duplo responderam adequadamente. A interface homem-máquina mostrou-se intuitiva: o display LCD atualizou a posição em tempo real, o teclado matricial permitiu programar destinos com dupla confirmação, e os botões manuais possibilitaram ajustes finos. A proteção por fim de curso atuou de forma eficaz, interrompido os comandos manuais quando o eixo atingia o limite de 180 graus, preservando a integridade do cabo da antena e do próprio encoder.

Com limitações identificadas, destaca-se a velocidade máxima de 3 RPM, imposta para evitar desgaste na integridade da antena. Por fim, o sistema desenvolvido pode ser integrado a uma antena real de 10 metros, automatizando seu posicionamento para máxima captação de sinal em comunicações de rádio amador, bem como adaptado para outras aplicações que exigem posicionamento angular preciso e de baixo custo, braços robóticos e mesas giratórias.

Dessa forma, o trabalho contribui para a disseminação do conhecimento sobre hardware e software de códigos abertos aplicados à mecatrônica, demonstrando que é possível desenvolver sistemas de precisão com componentes acessíveis e técnicas de manufatura aditiva, sem abrir mão da confiabilidade e da segurança operacional.[30]

30 O código-fonte completo, os esquemáticos e os arquivos STL das peças impressas estão disponíveis para consulta e reprodução, conforme os princípios de hardware e software de códigos abertos adotados no projeto.

# 6 TRABALHOS FUTUROS

Com base na experiência adquirida ao longo do desenvolvimento e testes deste protótipo, diversas oportunidades de aprimoramento e evolução do sistema se apresentam para trabalhos futuros. As sugestões a seguir visam não apenas o aperfeiçoamento técnico, mas também a viabilização comercial do produto, com redução de custos, miniaturização, maior robustez e melhor experiência de usuário.

A primeira e mais importante diretriz para trabalhos futuros é a integração e miniaturização dos subsistemas. O protótipo atual foi construído em uma estrutura de três andares com chapas de acrílico e madeira, utilizando módulos separados (placa de sensores, placa emissora, placa receptora, placa PWM, Arduino, relés, fonte ATX). Essa abordagem, embora adequada para validação de conceito e testes de bancada, resulta em um volume considerável e muitas interconexões com cabos, o que aumenta a suscetibilidade a ruídos e falhas de contato. Em versões futuras, propõe-se o desenvolvimento de uma única placa de circuito impresso (PCI) customizada que integre todos os subsistemas eletrônicos: o microcontrolador ATmega 2560 (ou uma versão mais compacta como o ATmega 328 ou ESP32), os oito pares de LED IR e fototransistor com seus resistores de polarização, a placa PWM com os dois CI 555 e o driver IR4427, o módulo de acionamento dos relés (ou ponte H MOSFET), o conector para o display, o conector para o teclado (ou touchscreen) e os terminais de alimentação. Essa unificação reduziria drasticamente a quantidade de cabos, eliminaria conectores DB intermediários, diminuiria o ruído eletromagnético e facilitaria a montagem em um invólucro compacto.

Como segunda sugestão, propõe-se a substituição da estrutura atual por um case industrial em aço inoxidável (inox) de dimensões reduzidas, que abrigue toda a eletrônica integrada em uma única carcaça. O inox foi escolhido por sua resistência mecânica, durabilidade, imunidade à corrosão e blindagem eletromagnética natural, protegendo os circuitos sensíveis (especialmente os fototransistores e o condicionamento de sinais) contra interferências externas. O case deve ser projetado com canaletas internas para passagem organizada dos cabos, suportes para fixação da placa-mãe, furos para dissipação térmica (já que o motor e os relés podem gerar calor) e um sistema de fechamento com juntas de silicone para proteção contra poeira e umidade, permitindo o uso do equipamento em ambientes industriais ou externos (como em uma estação de rádio). O tamanho alvo seria aproximadamente 20 cm x 15 cm x 8 cm, reduzindo o volume atual em cerca de 70%.

A terceira sugestão é a substituição do display LCD 20x4 e do teclado matricial 4x4 por uma interface touchscreen colorida, como um display TFT LCD com tela sensível ao toque (ex.:

Nextion ou ILI9341 com placa de toque resistiva). Um display de 3,5 polegadas ou 5 polegadas pode exibir, além da posição angular e do código Gray, gráficos de movimento, menus com botões virtuais, indicadores de limite, velocidade atual, destino programado e até mesmo um pequeno mostrador analógico circular. A interface touchscreen elimina a necessidade dos 16 botões físicos do teclado, reduzindo componentes, furação no case e pontos de falha mecânica. Os botões de giro manual (esquerda/direita) e o botão de reset podem ser mantidos como elementos físicos de segurança, mas podem também ser implementados como botões virtuais na tela, com a opção de confirmação por toque prolongado para evitar acionamentos acidentais. O touchscreen deve ser montado na parte frontal do case de inox, com uma moldura de silicone para vedação e proteção contra impactos.

A quarta sugestão é a migração do firmware para uma arquitetura baseada em interrupções e multitarefa, utilizando um microcontrolador mais moderno como o ESP32 (que possui dois núcleos, Wi-Fi e Bluetooth integrados, e frequência de até 240 MHz). Com o ESP32, seria possível implementar um servidor web embarcado para monitoramento remoto via smartphone ou computador, registro de dados em cartão SD, atualização de firmware over-the-air (OTA) e integração com sistemas de automação via MQTT ou Modbus TCP. Além disso, o segundo núcleo do ESP32 poderia ser dedicado exclusivamente à leitura dos sensores e ao controle do motor, garantindo tempo de resposta determinístico mesmo com a interface gráfica rodando em paralelo.

A quinta sugestão é o redimensionamento da fonte de alimentação. A fonte ATX 500W, embora robusta, é excessivamente grande para a versão miniaturizada. Propõe-se a utilização de uma fonte chaveada compacta de 12V/5A (60W) para o motor e os relés, combinada com um conversor buck para gerar os 5V necessários para o microcontrolador, sensores e touchscreen. Essa fonte pode ser embutida dentro do próprio case de inox, desde que sejam previstos ventilação e isolamento adequados. Caso o motor utilizado exija mais potência, pode-se optar por uma fonte externa (tipo fonte de notebook) conectada via conector P4, mantendo o case menor e mais leve.

A sexta sugestão diz respeito ao processo de manufatura e ao modelo de comercialização. Todo o projeto – incluindo os esquemáticos eletrônicos, o layout da PCI, o firmware do microcontrolador, o código da interface touchscreen, os arquivos STL das peças impressas (caso ainda utilizadas para algum componente), o modelo 3D do case de inox e a documentação técnica – será disponibilizado publicamente no GitHub sob uma licença de hardware livre (como CERN OHL ou TAPR OHL). Isso permite que a comunidade estude, modifique e aprimore o projeto, bem como que terceiros possam fabricar suas próprias unidades ou componentes. Para-

lelamente, o sistema também será comercializado em duas versões: (1) kit de componentes eletrônicos e mecânicos (placa PCI, case de inox, display touchscreen, motor, cabos) para montagem por hobistas e pequenas empresas; (2) produto final montado, testado e calibrado, com garantia e suporte técnico. O preço estimado de venda do produto acabado seria competitivo em relação a encoders absolutos comerciais de resolução equivalente, graças à economia de escala e ao uso de componentes de código aberto.[31]

31 O repositório GitHub conterá todos os arquivos necessários para reproduzir o sistema, incluindo esquemáticos (KiCad), firmware (Arduino), modelos 3D (Autodesk Inventor), instruções de montagem e lista de materiais. O produto comercializado terá garantia de 12 meses e suporte técnico via fórum dedicado.

# ANEXO A – Programação Rotor Absoluto

```
/*
  ROTOR ANTENA - VERSAO V5 ( Teclado + BigFont + Buzzer)
   - Foco em operação Manual e Presets via Teclado
*/
// ================== BIBLIOTECAS ==================
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
#include <BigFont01_I2C.h>

// ================== OBJETOS ==================
LiquidCrystal_I2C lcd(0x27, 20, 4);
BigFont01_I2C big(&lcd);

// ================== PINOS ==================
#define BTN_HORARIO 2
#define BTN_A_HORARIO 3
#define RELE_HORARIO 31
#define RELE_A_HORARIO 5
#define PIN_BUZZER 4
#define RELE_LIGADO LOW
#define RELE_DESLIGADO HIGH

const int grayPins[8] = { 22, 23, 24, 25, 26, 27, 28, 29 };

// ================== TECLADO ==================
const byte ROWS = 4, COLS = 4;
char keys[ROWS][COLS] = {
  { '1', '2', '3', 'A' },
  { '4', '5', '6', 'B' },
  { '7', '8', '9', 'C' },
  { '*', '0', '#', 'D' }
};
byte rowPins[ROWS] = { 6, 7, 8, 9 };
byte colPins[COLS] = { 10, 11, 12, 13 };
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

// ================== SISTEMA ==================
enum STATE { MANUAL,
          PROGRAMANDO,
```

```
        AUTO };  //estado
STATE estado = MANUAL;

float posDeg = 0;
float targetDeg = 0;
const float STEP_DEG = 360.0 / 256.0;
const float LIMIT = 180.0;  // BLOQUEIO/FIM DE CURSO
const float LIMIT_TOL = 1.5;

// ================= UTILIDADES MATEMÁTICAS ================
float norm360(float a) {
  while (a < 0) a += 360;
  while (a >= 360) a -= 360;
  return a;
}
float distCW(float from, float to) {
  return norm360(to - from);
}
float distCCW(float from, float to) {
  return norm360(from - to);
}

bool reachedLimit() {
  return abs(norm360(posDeg) - LIMIT) < LIMIT_TOL;
}

bool arcCrossLimit(float start, float end, bool cw) {
  float s = norm360(start), e = norm360(end), L = LIMIT;
  if (cw) {
    float d = distCW(s, e), dLim = distCW(s, L);
    return dLim > 0 && dLim < d;
  } else {
    float d = distCCW(s, e), dLim = distCCW(s, L);
    return dLim > 0 && dLim < d;
  }
}

// ================= CONTROLE MOTOR ================
void stopMotor() {
  digitalWrite(RELE_HORARIO, RELE_DESLIGADO);
  digitalWrite(RELE_A_HORARIO, RELE_DESLIGADO);
}
void moveHORARIO() {
```

```
  digitalWrite(RELE_A_HORARIO, RELE_DESLIGADO);
  digitalWrite(RELE_HORARIO, RELE_LIGADO);
}
void moveAHORARIO() {
  digitalWrite(RELE_HORARIO, RELE_DESLIGADO);
  digitalWrite(RELE_A_HORARIO, RELE_LIGADO);
}

// ================= LEITURA GRAY =================
unsigned int grayToBinary(unsigned int num) {
  for (unsigned int mask = num >> 1; mask != 0; mask >>= 1) num ^= mask;
  return num;
}
void readPosition() {
  byte g = 0;
  for (int i = 0; i < 8; i++) {
    if (digitalRead(grayPins[i])) bitSet(g, i);
  }
  int idx = grayToBinary(g);
  posDeg = idx * STEP_DEG;
}

// ================= INTERFACE LCD =================
void drawBase() {
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("Rotor de Antena");
  lcd.setCursor(14, 2);
  lcd.print("D:---");
}

void showPos() {
  lcd.setCursor(0, 2);
  lcd.print("P:");
  int posInt = (int)posDeg;
  big.writeint(1, 3, posInt, 3, true);
  lcd.setCursor(12, 1);
  lcd.print("o");
}

// ================= LÓGICA MANUAL =================
void controlManual() {
  bool btnCw = (digitalRead(BTN_HORARIO) == LOW);
```

```
  bool btnCcw = (digitalRead(BTN_A_HORARIO) == LOW);
  bool noLimite = reachedLimit();
  bool bCW = false, bCCW = false;

  if (noLimite) {
    if (posDeg <= LIMIT) bCW = true;
    else bCCW = true;
  }

  if (btnCw && !bCW) {
    moveHORARIO();
    lcd.setCursor(0, 3);
    lcd.print("M: Horario       ");
  } else if (btnCcw && !bCCW) {
    moveAHORARIO();
    lcd.setCursor(0, 3);
    lcd.print("M: Anti-Hor      ");
  } else {
    stopMotor();
    if ((btnCw && bCW) || (btnCcw && bCCW)) {
      lcd.setCursor(0, 3);
      lcd.print("BLOQUEIO LIMITE   ");
      emitirSom(3);  //som que avisa do bloqueio
    } else {
      lcd.setCursor(0, 3);
      lcd.print("Status: Parado    ");
    }
  }
}

// ================= LÓGICA AUTOMÁTICA =================
void startAuto(float dest) {
  targetDeg = norm360(dest);
  lcd.setCursor(16, 2);
  if (targetDeg < 100) lcd.print("0");
  if (targetDeg < 10) lcd.print("0");
  lcd.print((int)targetDeg);

  bool cwCross = arcCrossLimit(posDeg, targetDeg, true);
  bool ccwCross = arcCrossLimit(posDeg, targetDeg, false);
  bool chooseCW;

  if (distCW(posDeg, targetDeg) <= distCCW(posDeg, targetDeg)) {
```

```
    chooseCW = !cwCross;
  } else {
    chooseCW = ccwCross;
  }

  if (chooseCW) moveHORARIO();
  else moveAHORARIO();
  estado = AUTO;
}

unsigned long tempoMensagem = 0;
bool aguardandoLimpeza = false;

void controlAuto() {
  float diff = abs(norm360(posDeg - targetDeg));

  if (!aguardandoLimpeza) {  // Enquanto o motor está girando...
    if (diff < STEP_DEG || reachedLimit()) {
      stopMotor();
      if (reachedLimit()) emitirSom(3);
      else emitirSom(2);  //  bi-bi

      lcd.setCursor(0, 3);
      lcd.print(reachedLimit() ? "ERRO: FIM CURSO   " : "Destino OK!       ");

      tempoMensagem = millis();  // Marca quando a mensagem apareceu
      aguardandoLimpeza = true;  // Avisa que precisamos limpar o LCD depois
    } else {
      lcd.setCursor(0, 3);
      lcd.print("Auto -> ");
      lcd.print((int)targetDeg);
      lcd.print(" deg     ");
    }
  } else {  // Se já chegou, espera 2 segundos sem travar o código
    if (millis() - tempoMensagem > 2000) {
      drawBase();
      estado = MANUAL;
      aguardandoLimpeza = false;
    }
  }
}

// ================= MODO TECLADO =================
```

```
void modoProgramacao() {
  estado = PROGRAMANDO;
  stopMotor();
  lcd.clear();
  lcd.print("Novo Destino:");
  lcd.setCursor(0, 2);
  lcd.print("Digito: ___");

  String entrada = "";
  while (estado == PROGRAMANDO) {
    gerenciarSons();
    char k = keypad.getKey();
    if (k != NO_KEY) {
      emitirSom(1);  // <-- para cada número bipar!
      if (k >= '0' && k <= '9' && entrada.length() < 3) {
        entrada += k;
        lcd.setCursor(8, 2);
        lcd.print(entrada);
      }
      if (k == '*') {
        entrada = "";
        lcd.setCursor(8, 2);
        lcd.print("___");
      }
      if (k == '#' && entrada.length() > 0) {
        int val = entrada.toInt();
        if (val >= 0 && val <= 360) {
          drawBase();
          startAuto(val);
          return;
        }
      }
      if (k == 'D') {  // Cancelar
        drawBase();
        estado = MANUAL;
      }
    }
  }
}

// ================= SONS ================
unsigned long tempoSom = 0;
int faseSom = 0;
```

```
void emitirSom(int tipo) {
  if (tipo == 1) {
    tone(PIN_BUZZER, 2000, 40);
  } else if (tipo == 2) {
    tone(PIN_BUZZER, 1500, 30);
    tempoSom = millis();
    faseSom = 1;
  } else if (tipo == 3) {
    tone(PIN_BUZZER, 450, 600);
  }
}

void gerenciarSons() {
  if (faseSom == 1 && (millis() - tempoSom > 150)) {
    tone(PIN_BUZZER, 1500, 30);
    faseSom = 0;
  }
}

// ================== SETUP ==================
void setup() {
  lcd.init();
  lcd.backlight();
  big.begin();

  pinMode(BTN_HORARIO, INPUT_PULLUP);
  pinMode(BTN_A_HORARIO, INPUT_PULLUP);
  pinMode(RELE_HORARIO, OUTPUT);
  pinMode(RELE_A_HORARIO, OUTPUT);
  pinMode(PIN_BUZZER, OUTPUT);

  for (int i = 0; i < 8; i++) pinMode(grayPins[i], INPUT);

  stopMotor();
  drawBase();
}

// ================== LOOP PRINCIPAL ==================
void loop() {
  readPosition();
  showPos();
  gerenciarSons();
```

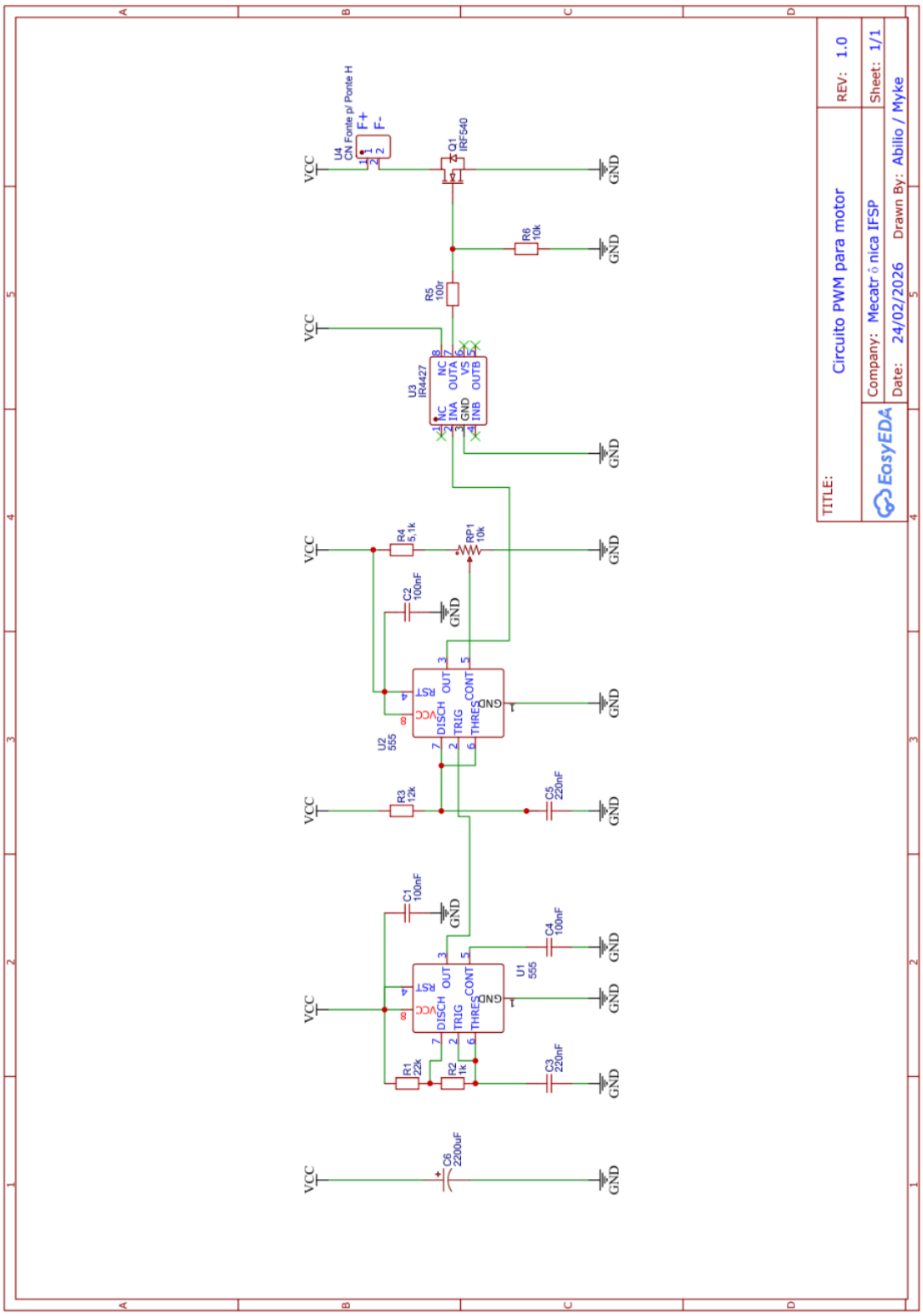
```
  char key = keypad.getKey();
  if (key != NO_KEY) {
    emitirSom(1);
  }

  if (estado == MANUAL) {
    if (key == 'A') startAuto(330.0);
    if (key == 'B') startAuto(45.0);
    if (key == 'C') startAuto(120.0);
    if (key == 'D') startAuto(170.0);
    if (key == '*') modoProgramacao();

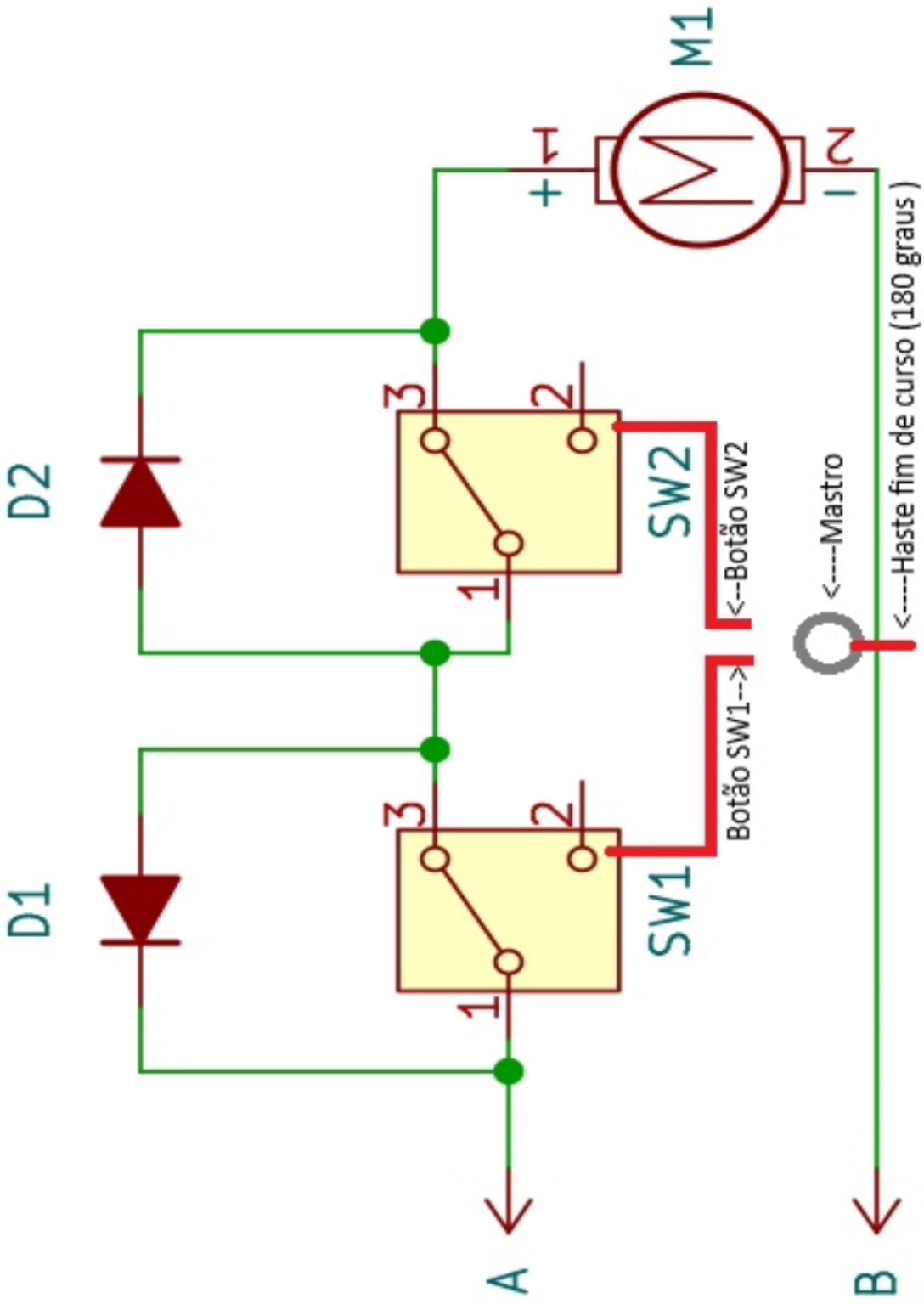
    controlManual();
  }

  if (estado == AUTO) controlAuto();
}
```

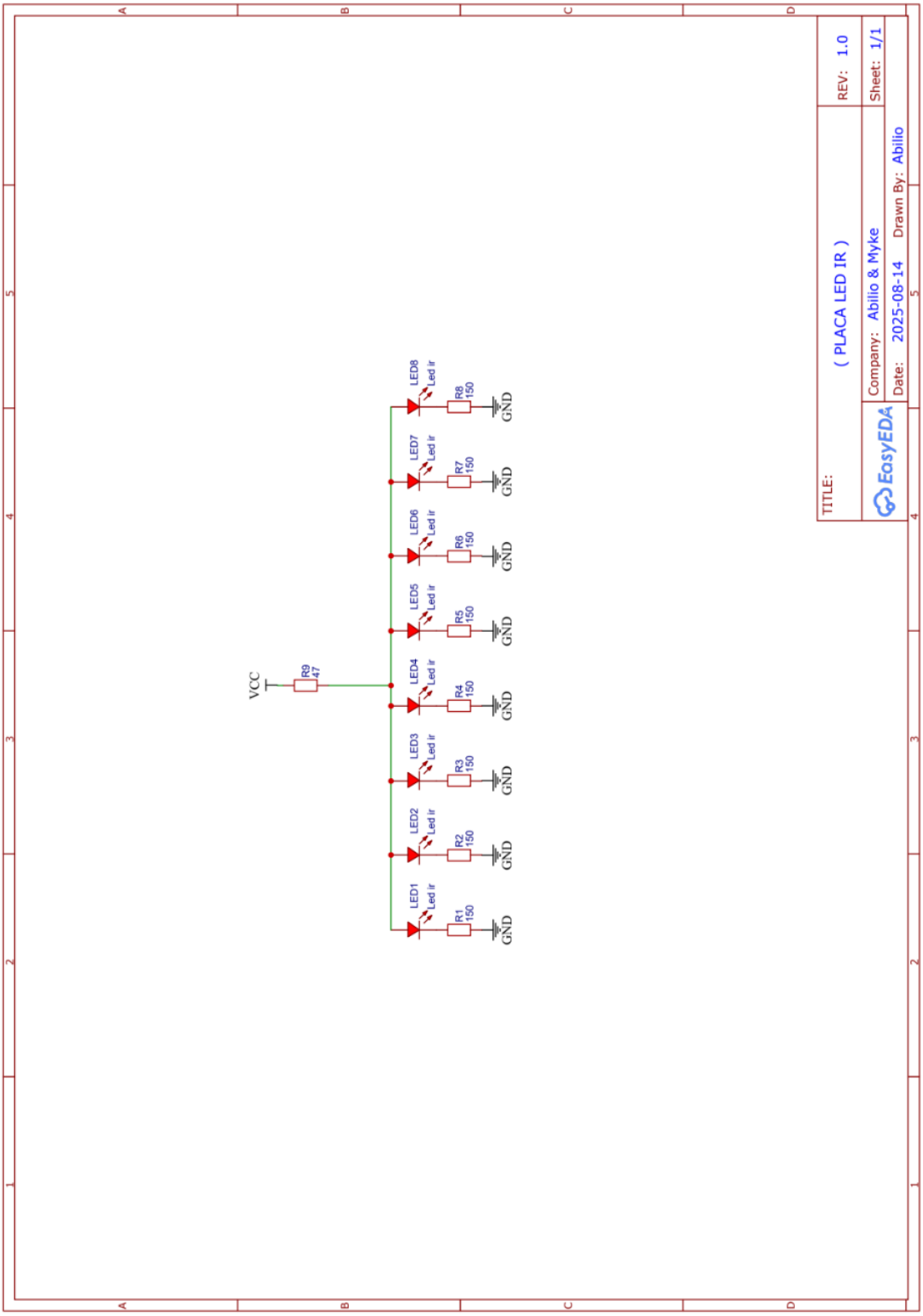
# ANEXO B – Esquema Eletrônico PWM CI 555 e IR4427



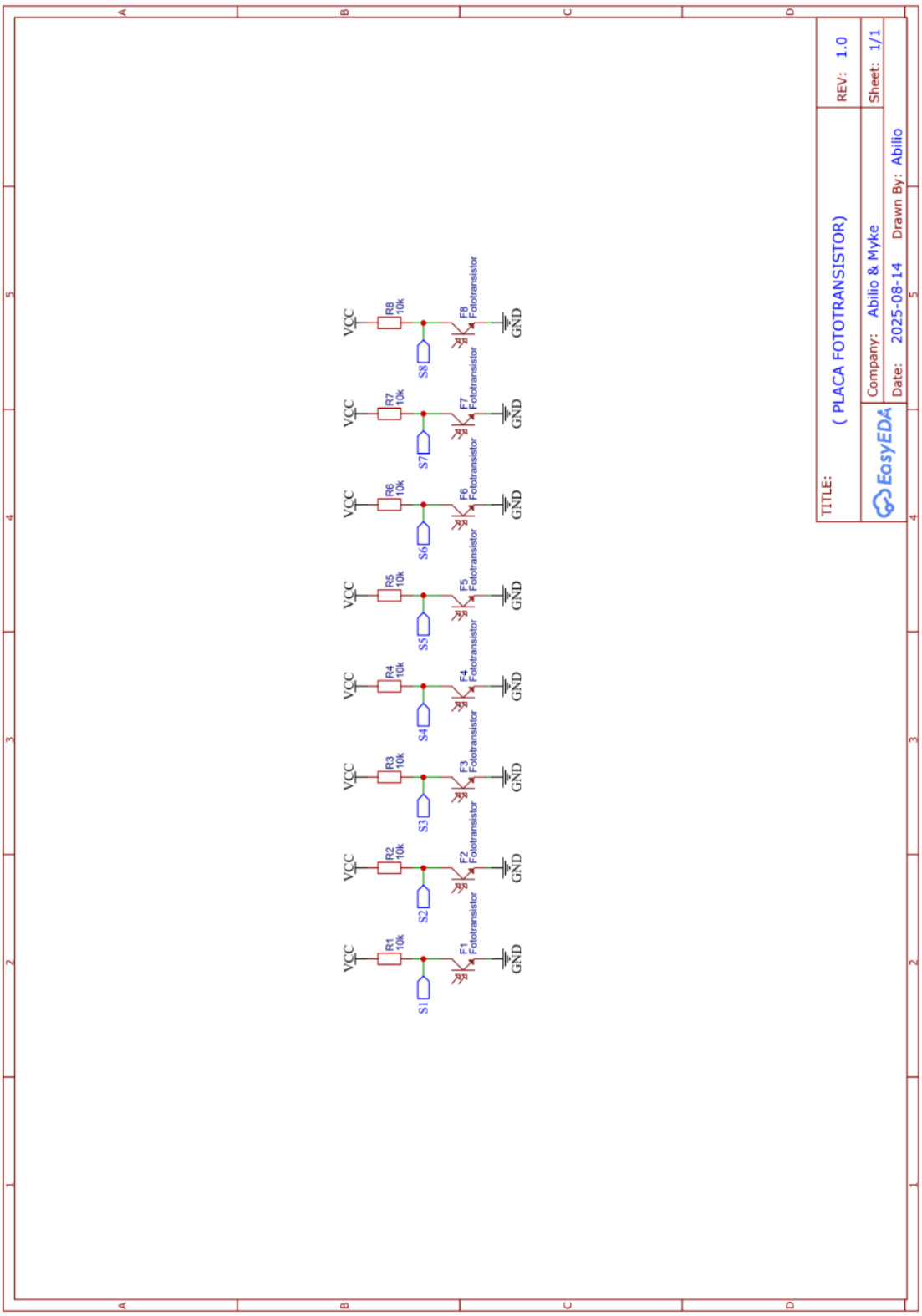
**ANEXO C – Esquema elétrico da proteção com diodos D1 e D2**



# ANEXO D – Esquema Elétrico Emissor



# ANEXO E – Esquema Elétrico Receptor



# ANEXO F – Esquema Elétrico de Interface

